



TRADINGPRO

Eksamensrapport

Udvikling af en realtidsplatform til analyse af kryptovalutamarkedet

Ozan Korkmaz

Vejleder: Flemming Sørensen

1205hf26086per

03-09-2026

Indholdsfortegnelse

1. Forord.....	3
2. Hvem er jeg?	3
3. Indledning.....	3
4. Problemformulering	3
5. Projektbeskrivelse.....	4
6. Kravspecifikation	4
7. Projektplanlægning og udviklingsforløb.....	5
8. Teknologier.....	5
9. Systemarkitektur	7
10. Backend	8
11. Klasseoverblik	9
12. Exchange-arkitektur.....	10
13. Historical market data	11
14. Realtime market data.....	12
15. Databaseoverblik.....	13
16. Database og Entity Framework Core	14
17. Modeloverblik	14
18. TimescaleDB og timeframes	15
19. Candle API og lazy loading	16
20. Frontend	17
21. Candlestick-chart og analyseværktøjer	19
22. Watchlists	19
23. Alerts	19
24. AI-analyse	20
25. Brugerindstillinger og internationalisering.....	20
26. Administration.....	20
27. Authentication og sikkerhed	21
28. Kommunikation mellem frontend og backend.....	22
29. Fejlhåndtering og robusthed	23
30. Teststrategi.....	24
31. Testplan.....	25
32. Testresultater.....	26
33. Performance og skalerbarhed	26
34. Designvalg og kodekvalitet	27
35. Brugergænseflade og UX	28
36. Brugervejledning.....	29
37. Afprøvning og fejlsøgning	34
38. Udviklingslogbog.....	35
39. Projektets videreudvikling	35
40. Konklusion	36
41. Bilagsoversigt	37

1. Forord

TradingPro er mit afsluttende projekt og er udviklet som en samlet webapplikation til overvågning og analyse af kryptovalutamarkedet. Projektet samler frontend, backend, database, realtidskommunikation, eksterne exchange-API'er, sikkerhed, administration og automatiserede tests i en løsning.

Rapporten er skrevet, så den både beskriver det færdige produkt og forklarer de tekniske valg bag løsningen. Tekniske begreber forklares løbende, så læseren ikke behøver et dybt kendskab til programmering for at følge dataflowet fra børsen til chartet.

1.1 Formål med rapporten

Formålet er at dokumentere, hvordan TradingPro er planlagt, opbygget, implementeret og afprøvet. Rapporten beskriver de centrale krav, arkitekturen, market-data flowet, databasen, frontendens funktioner, sikkerhed, test og de erfaringer, der blev gjort under udviklingen.

2. Hvem er jeg?

Jeg hedder Ozan Korkmaz og er elev på EUD-uddannelsen på TEC. Jeg har været i gang med uddannelsen i cirka 4,5 år og har gennem uddannelsen og min læreplads fået erfaring med blandt andet C#, JavaScript, Python, ABAP, HTML og CSS.

Jeg arbejder som Full-Stack udvikler, men jeg interesserer mig mest for backend- og API-udvikling. Jeg kan godt lide at arbejde med den del af et system, der ligger bag brugergrænsefladen, for eksempel databaser, integrationer, API'er og den logik, der får forskellige dele af en løsning til at arbejde sammen.

Siden jeg startede i lære hos KMD A/S, har jeg hovedsageligt arbejdet med backend-relaterede opgaver og softwareudvikling. Det har også haft betydning for mit valg af TradingPro som afsluttende projekt, fordi projektet gav mig mulighed for at arbejde videre med områder, jeg allerede havde interesse for, men samtidig udfordre mig selv med blandt andet realtime-data, frontend og integration til flere exchanges.

3. Indledning

Kryptomarkedet producerer nye prisdata døgnet rundt. En analyseplatform skal derfor kunne kombinere store mængder historiske data med løbende live-opdateringer. Hvis kun den aktuelle pris vises, mangler brugeren konteksten; hvis kun historiske data vises, mister platformen sin realtime-værdi.

TradingPro løser dette ved at hente market data fra eksterne exchanges, gemme grunddata lokalt, aggregere data til flere timeframes og sende aktuelle ændringer til browseren. Samtidig giver applikationen brugeren funktioner som watchlists, chart-tegninger, alerts, AI-analyse og personlige indstillinger.

4. Problemformulering

Formålet med TradingPro er at undersøge, hvordan jeg kan udvikle en webbaseret tradingplatform, som kan håndtere både historiske og realtime markedsdata på en effektiv og overskuelig måde.

En central del af projektet er at opbygge en arkitektur med ASP.NET Core, React, PostgreSQL og TimescaleDB, hvor store mængder finansielle tidsseriedata kan hentes fra eksterne exchanges, gemmes i databasen og efterfølgende visualiseres i et interaktivt chart.

Projektet undersøger samtidig, hvordan eksterne AI-tjenester kan integreres i platformen for at understøtte tekniske analyser af udvalgte kryptovalutaer, og hvordan adgang til systemets funktioner kan sikres gennem authentication og rollebaseret adgangskontrol med rollerne User, Admin og SuperAdmin.

På den baggrund arbejder jeg med følgende spørgsmål:

- Hvordan kan jeg opbygge en backend og databasearkitektur, der effektivt kan håndtere historical og realtime market data?
- Hvordan kan markedsdata præsenteres i en React-baseret brugergrænseflade med interaktive charts og forskellige timeframes?
- Hvordan kan eksterne exchanges og AI-tjenester integreres uden at gøre resten af systemet afhængigt af en bestemt leverandør?
- Hvordan kan authentication og rollebaseret adgangskontrol beskytte bruger- og administrationsfunktioner?

5. Projektbeskrivelse

TradingPro består af et dashboard, hvor brugeren vælger exchange, symbol og timeframe og derefter arbejder med et candlestick-chart. Backend fungerer som mellemlid mellem browseren, databasen og de eksterne exchanges. Det betyder, at frontend ikke behøver kende Binance- eller OKX-specifikke dataformater.

5.1 Målgruppe

Løsningen er rettet mod brugere, som ønsker et samlet overblik over kryptomarkedet og vil kunne analysere prisudvikling uden at skifte mellem flere værktøjer. Systemet er ikke udviklet som børs eller broker og gennemfører ikke handler på brugerens vegne.

5.2 Afgrænsning

Automatiseret køb og salg, ordreafgivelse og opbevaring af kryptovaluta er uden for projektets scope. Fokus er dataindsamling, visualisering, analyse, brugerfunktioner og administration.

6. Kravspecifikation

Bilag 1 indeholder den oprindelige kravspecifikation. Den dannede grundlag for implementeringen og beskrev blandt andet login, candlestick-chart, historik, live-data, AI-analyse og rollebaseret adgang. Under udviklingen blev løsningen udvidet på flere områder, uden at hovedformålet blev ændret.

Område	Oprindeligt fokus	Implementeret løsning
Exchange	Binance	Binance og OKX via fælles exchange-kontrakt
Timeframes	Kortere perioder	1m, 3m, 5m, 15m, 30m, 1h, 2h, 4h, 1d, 1w, 1mon og 1y
Watchlist	Personlig overvågning	CRUD af lister og symboler samt live-pris/ændring
Chart	Candlestick-visning	Lazy loading, realtime-opdatering og gemte tegninger
Admin	Roller og symboler	Brugere, symboler, systemstatus og exchanges
Test	xUnit	39 automatiserede repository- og controller-tests

7. Projektplanlægning og udviklingsforløb

Udviklingen blev gennemført iterativt. I stedet for at implementere hele systemet på en gang blev funktionerne bygget og afprøvet i mindre trin. Denne arbejdsform var særlig vigtig for market data, fordi fejl kunne opstå flere steder i samme kæde: exchange, historisk synkronisering, database, continuous aggregates, API, SignalR eller frontend.

7.1 Centrale udviklingsfaser

Projektet blev udviklet i flere faser, hvor funktionerne løbende blev implementeret, testet og videreudviklet.

Fase	Fokus
1	Datamodel, brugerhåndtering og grundlæggende API
2	Exchange-integration og historical market data
3	Candlestick-chart og frontend-dashboard
4	Realtime WebSocket og SignalR
5	TimescaleDB, timeframes og lazy loading
6	Watchlists, alerts, chart drawings og AI-panel
7	Adminfunktioner og OKX-integration
8	Automatiserede tests, fejlfinding og dokumentation

7.2 Versionskontrol

Git blev brugt til versionsstyring. Det gjorde det muligt at arbejde med større ændringer som databaseopsætning, chart-loading og exchange-integration uden at miste tidligere fungerende kode. Versionskontrol gav samtidig et historisk overblik over ændringer og gjorde fejlretning mere kontrollerbar.

8. Teknologier

TradingPro er bygget med forskellige teknologier, som hver har en bestemt rolle i systemet. Tabellen nedenfor viser de vigtigste teknologier og deres formål.

Område	Teknologi	Formål
Frontend	React + TypeScript + Vite	Komponentbaseret brugerflade, typekontrol og build
Backend	ASP.NET Core / C#	REST API, services, sikkerhed, sync og SignalR
Database	PostgreSQL + TimescaleDB	Relationelle data og tidsserier
ORM	Entity Framework Core	Mapping mellem C#-modeller og database
Realtime	WebSocket + SignalR	Exchange-data ind og live-opdateringer ud
Test	xUnit + EF Core InMemory + Moq	Automatiseret kontrol af repositories og controllers
Exchanges	Binance + OKX	Historiske og aktuelle kryptodata

8.1 Hvorfor denne kombination?

Jeg har valgt teknologierne ud fra, hvad de skulle løse i projektet. Nogle kendte jeg i forvejen, mens andre blev valgt, fordi TradingPro arbejder med store mængder tidsbaserede data og realtime-opdateringer. Her forklarer jeg kort, hvad de vigtigste teknologier er, hvordan jeg bruger dem, og hvorfor jeg valgte dem.

8.2 ASP.NET Core og C#

ASP.NET Core er frameworket, jeg bruger til backend og REST API, mens C# er programmeringssproget. Jeg valgte kombinationen, fordi den passer godt til controllers, services, repositories, dependency injection og SignalR. Det gjorde det muligt for mig at holde API, forretningslogik og databaseadgang adskilt.

8.3 Entity Framework Core

Entity Framework Core er et ORM-værktøj til .NET. Det betyder, at jeg kan arbejde med databasen gennem C#-klasser og LINQ i stedet for at skrive al SQL manuelt. I TradingPro bruger jeg det blandt andet til users, exchanges, symbols, watchlists og andre relationelle data. Jeg valgte det, fordi det passer naturligt sammen med ASP.NET Core og gør databasekoden lettere at vedligeholde.

8.4 PostgreSQL

PostgreSQL er den relationelle database, som TradingPro bygger på. Her gemmer jeg blandt andet users, exchanges, symbols, candles, watchlists, alerts og chart drawings. Jeg valgte PostgreSQL, fordi det er stabilt og modent, men især fordi TimescaleDB kan køre oven på PostgreSQL.

8.5 TimescaleDB

TimescaleDB er en udvidelse til PostgreSQL, som er lavet til tidsseriedata. Candles passer godt til denne type database, fordi hver candle hører til et bestemt tidspunkt. Jeg bruger TimescaleDB til historical market data, hypertable og større timeframes gennem continuous aggregates. Det var en vigtig grund til valget.

8.6 React

React er biblioteket, jeg bruger til brugergrænsefladen. Jeg kan dele siden op i komponenter som chart, watchlist, alerts, AI-panel og administration. Jeg valgte React, fordi TradingPro har mange dele af skærmen, der ændrer sig dynamisk, og fordi komponenter gør frontend lettere at udvide.

8.7 TypeScript

TypeScript bygger videre på JavaScript og tilføjer typer. Det hjælper mig med at opdage fejl tidligere, for eksempel hvis et API-response ikke har den form, frontend forventer. I TradingPro bruger jeg typer til blandt andet candles, symbols, exchanges og watchlists.

8.8 Vite

Vite bruges som build- og udviklingsværktøj til frontend. Det starter udviklingsmiljøet hurtigt og bygger React/TypeScript-projektet til browseren. Jeg valgte det, fordi det er enkelt og passer godt til et moderne React-projekt.

8.9 Tailwind CSS

Tailwind CSS er et CSS-framework, hvor styling bygges op med små utility-klasser direkte i komponenterne. Jeg bruger det til at holde TradingPros brugergrænseflade ensartet og responsiv på tværs af chart, paneler, formularer og administration. Jeg valgte Tailwind CSS, fordi det gjorde det hurtigt at justere layout og spacing uden at skulle oprette en stor mængde separate CSS-klasser.

8.10 TradingView Lightweight Charts

TradingView Lightweight Charts er et let JavaScript-bibliotek til finansielle grafer. I TradingPro bruger jeg det til candlestick-chartet og visualisering af prisdata. Jeg valgte biblioteket, fordi det er lavet specifikt til finansielle tidsserier, har god performance ved mange datapunkter og giver de nødvendige funktioner til zoom, scrolling og realtime-opdatering uden at være et komplet TradingView-produkt.

8.11 REST API

Et REST API er den del af backend, som frontend sender almindelige HTTP-requests til. I TradingPro bruger jeg REST til blandt andet login, candles, watchlists, alerts, settings og administration. Det passer godt til handlinger, hvor klienten selv starter requesten og forventer et svar.

8.12 WebSocket og SignalR

WebSocket gør det muligt at holde en forbindelse åben og modtage nye data løbende. Exchanges leverer realtime market data gennem WebSocket. Mellem min egen backend og frontend bruger jeg SignalR, fordi det gør realtime-kommunikation og grupper pr. exchange/symbol lettere at håndtere.

8.13 JWT

JWT bruges til authentication mellem frontend og backend. Efter login får klienten et token, som sendes med beskyttede API-kald. Backend kan derefter se, hvilken bruger der laver requesten, og hvilken rolle brugeren har.

8.14 xUnit, Moq og EF Core InMemory

xUnit er test-frameworket i testprojektet. Moq bruges til at erstatte afhængigheder med kontrollerede test-objekter, og EF Core InMemory bruges til repository-tests uden den rigtige PostgreSQL-database. Det gør testene hurtige og isolerede.

9. Systemarkitektur

Systemet er opdelt i fire hoveddele: frontend, backend, database og eksterne datakilder. Opdelingen reducerer kobling. Frontend arbejder med TradingPros egne DTO'er og endpoints; exchange-specifikke detaljer holdes i backendens adaptere.

Jeg har valgt denne opdeling, fordi det gør det lettere at finde ud af, hvor en opgave hører hjemme. Frontend skal først og fremmest vise data og håndtere brugerens handlinger. Backend skal validere requests, udføre logik og styre integrationerne. Databasen skal gemme data stabilt, mens Binance og OKX betragtes som eksterne datakilder. Hvis en exchange ændrer sit API-format, er målet derfor, at ændringen hovedsageligt kan håndteres i exchange-laget uden at chartet skal bygges om.

Del	Ansvar
React frontend	Viser chart og paneler, sender REST-kald og modtager SignalR-events
ASP.NET Core backend	Validerer requests, koordinerer services, håndterer auth, market data og AI
PostgreSQL/TimescaleDB	Gemmer brugerdata, symboler, candles og aggregerede timeframes
Binance/OKX	Leverer historical data via REST og aktuelle data via WebSocket

9.1 Dataflow

TradingPro har to centrale dataflows for market data: historical data og realtime-data.

Historical data følger typisk dette flow:

Exchange REST → ExchangeService → MarketDataSyncService → CandleRepository → TimescaleDB → Candle API → frontend

Her hentes tidligere candles fra exchange og gemmes lokalt, så frontend senere kan hente historikken gennem TradingPros eget API.

Realtime-data følger et andet flow:

Exchange WebSocket → ExchangeService → TradingPro backend → MarketDataHub / SignalR → frontend

Her modtager backend løbende nye markedsopdateringer og sender dem videre til de relevante klienter. Diagrammer over systemarkitekturen og dataflowet findes i **Bilag 2**.

9.2 REST, WebSocket og SignalR

TradingPro anvender **REST, WebSocket og SignalR** til forskellige typer kommunikation.

REST bruges, når frontend aktivt sender et request og forventer et response, eksempelvis ved hentning af historical candles, watchlists eller brugerdata.

WebSocket bruges mellem exchanges og TradingPros backend. Forbindelsen holdes åben, så Binance og OKX løbende kan sende nye markedsdata uden gentagne REST-requests.

SignalR bruges mellem TradingPros backend og frontend. Når backend modtager nye realtime-data, kan de sendes direkte videre til de relevante brugere og charts.

10. Backend

Backend er udviklet i C# med ASP.NET Core og er opdelt i Controllers, Services og Repositories. Hvert lag har sit eget ansvar, så HTTP-håndtering, forretningslogik og databaseadgang ikke blandes sammen.

Backend er den centrale del af TradingPro. Det er her, jeg samler reglerne for applikationen og sørger for, at frontend ikke behøver at kende detaljerne i databasen eller de eksterne exchange-API'er. Når frontend for eksempel beder om candles, går requesten ind gennem en controller, videre til den relevante service og derefter til repository/database. Svaret sendes tilbage i et format, som frontend kan arbejde direkte med.

Jeg har bevidst undgået at lægge al logik direkte i controllers. Det ville virke i en lille prototype, men ville hurtigt gøre projektet svært at teste og udvide. Den lagdelte struktur betyder også, at samme service- eller repositorylogik kan genbruges fra flere steder i systemet.

10.1 Controller-laget

Controllers er indgangen til REST API'et. De modtager requests, læser parametre og brugeroplysninger, kalder de relevante services og returnerer HTTP-statuskoder og data til klienten. Controlleren bør være forholdsvis tynd, fordi den tunge logik hører hjemme i services.

Et eksempel er en request fra chartet. Controlleren modtager symbol, timeframe, limit og eventuelt endTime, kontrollerer requesten og sender arbejdet videre. Controllerens ansvar er altså HTTP-delen - ikke selv at beregne timeframes eller skrive databaseforespørgsler.

10.2 Service-laget

Services indeholder applikationens forretningslogik og koordinerer flere afhængigheder.

MarketDataSyncService er et eksempel: den skal forstå exchange, symbol, interval, batches, historisk status og lagring, men selve SQL-adgangen delegeres til repository-laget.

Service-laget er derfor det sted, hvor de forskellige dele bliver bundet sammen. Her kan jeg for eksempel beslutte, om et symbol skal historical-synkroniseres, hvorfra synkroniseringen skal fortsætte, hvilken exchange-service der skal bruges, og hvornår realtime-flowet kan starte. På den måde ligger selve arbejdsgangen et sted i stedet for at være spredt mellem controller og databasekode.

Et andet konkret eksempel er ExchangeManagementService. Servicen håndterer oprettelse, opdatering og sletning af exchanges, kontrollerer dublerede exchange-koder og sikrer, at en exchange med tilknyttede symbols ikke slettes permanent. Selve databaseoperationerne delegeres til ExchangeRepository.

10.3 Repository-laget

Repositories samler databaseadgangen. Det gør forespørgsler og persistens genbrugelige og gør det lettere at teste logikken uden at starte hele applikationen. CandleRepository har et særligt ansvar, fordi candle-forespørgsler også skal tage højde for interval, endTime og TimescaleDB-views.

Repository-laget skjuler dermed, hvordan data konkret bliver hentet og gemt. En service kan bede om de seneste candles eller om tidspunktet for den seneste gemte candle uden selv at kende SQL-detajlerne. Det har været særligt nyttigt i TradingPro, fordi candle-data både findes som 1m-grunddata og som aggregerede timeframes.

10.4 Interfaces og dependency injection

Interfaces beskriver kontrakter mellem lagene. Dependency injection betyder, at en klasse får sine afhængigheder leveret udefra i stedet for selv at oprette dem. Det reducerer kobling og gør det muligt at erstatte en rigtig dependency med en test-double under unit tests.

I praksis registrerer jeg implementationerne i dependency injection-containeren, og ASP.NET Core leverer dem til de klasser, der har brug for dem. Det gør blandt andet IExchangeService, repositories og andre services lettere at udskifte eller mocke i tests. For mig har det også gjort projektstrukturen mere overskuelig, fordi afhængighederne fremgår tydeligt af konstruktørerne.

10.5 DTO'er

DTO'er bruges til at styre de data, der sendes mellem klient og API. Det forhindrer, at database-entiteter automatisk bliver API-kontrakter, og gør det lettere at validere input og ændre intern implementering uden at ændre hele frontend.

11. Klasseoverblik

TradingPro indeholder mange klasser, men de har ikke alle samme rolle. For at gøre backend lettere at forstå har jeg samlet de vigtigste klassetyper i et overblik. Det viser, hvordan en request typisk bevæger sig gennem systemet, og hvor de forskellige ansvarsområder ligger.

Lag / type	Eksempler	Ansvar i TradingPro
Controllers	AuthController, candle-, watchlist- og admin-controllers	Modtager HTTP-requests, validerer input på API-niveau og returnerer svar og statuskoder.
Services	MarketDataSyncService, ExchangeManagementService, exchange-services og øvrige applikationsservices	Indeholder arbejdsgange og forretningslogik og koordinerer repositories, exchanges og realtime-flow.

Repositories	CandleRepository, WatchlistRepository, ExchangeRepository og øvrige repositories	Henter og gemmer data og holder databaseforespørgsler væk fra controller- og service-laget.
Exchange-lag	IExchangeService, Binance-service, OKX-service og ExchangeServiceFactory	Skjuler forskellene mellem de eksterne exchanges og oversætter deres data til TradingPros fælles format.
Realtime	MarketDataHub og stream/runtime-komponenter	Sender live-opdateringer til de relevante klienter og holder styr på aktive realtime-forbindelser.
Data og modeller	AppDbContext, User, Exchange, Symbol, Candle, Watchlist m.fl.	Repræsenterer de data, som applikationen arbejder med, og relationerne til databasen.

Det vigtigste for mig har været, at en klasse så vidt muligt har et tydeligt ansvar. Når jeg for eksempel ændrer måden candles hentes fra databasen på, skal ændringen primært ligge i repository-laget. Hvis en exchange ændrer sit dataformat, skal ændringen primært ligge i exchange-adapteren. Det gør koden lettere at finde rundt i og mindsker risikoen for, at en ændring påvirker dele af systemet, som ikke burde blive berørt.

12. Exchange-arkitektur

TradingPro understøtter flere exchanges gennem en fælles IExchangeService-kontrakt. Formålet er at skjule forskellene mellem de eksterne exchange-API'er, så resten af market-data systemet kan arbejde med symboler og candles på en ensartet måde.

Hver exchange har sin egen implementation af kontrakten. Exchange-specifikke forskelle som endpoints, symbolformater og response-data håndteres derfor i exchange-laget i stedet for at blive spredt ud i resten af applikationen.

12.1 Binance

Binance-integrationen implementerer IExchangeService og håndterer kommunikationen med Binance. Den bruges blandt andet til at hente symbolinformation og historical candles samt modtage realtime market data gennem WebSocket.

Data fra Binance oversættes til TradingPro's fælles modeller, før de sendes videre til resten af systemet. Backend behøver derfor ikke arbejde direkte med Binance-specifikke response-formater.

12.2 OKX

OKX er implementeret som en separat exchange-service, men følger den samme IExchangeService-kontrakt som Binance.

De to exchanges har forskellige API'er og dataformater. Et simpelt eksempel er symbolformatet, hvor OKX kan bruge BTC-USDT, mens Binance typisk bruger BTCUSDT.

Denne forskel håndteres i OKX-adapteren. Resten af TradingPro kan derfor arbejde med market data gennem den samme struktur uden at skulle kende de enkelte exchanges interne formater.

12.3 ExchangeServiceFactory

ExchangeServiceFactory har ansvaret for at vælge den korrekte IExchangeService ud fra exchange-koden.

Hvis systemet eksempelvis arbejder med BINANCE, vælges Binance-servicen, mens OKX vælger OKX-servicen. Resten af market-data flowet behøver derfor ikke selv at afgøre, hvilken konkret implementation der skal bruges.

Denne struktur gør det lettere at tilføje flere exchanges senere. En ny exchange kan implementere den samme kontrakt og registreres i systemet uden at hele market-data flowet skal omskrives.

12.4 Exchange-administration

Det er vigtigt at skelne mellem market-data integration og administration af exchanges.

IXchangeService, Binance-, OKX-servicen og ExchangeServiceFactory håndterer kommunikationen med de eksterne exchanges. Oprettelse, opdatering og sletning af exchange-konfigurationen i TradingPros egen database håndteres derimod gennem:

ExchangesController → ExchangeManagementService → ExchangeRepository → AppDbContext

Denne opdeling betyder, at market-data integration og databaseadministration har hvert sit ansvar, selv om begge arbejder med exchanges.

13. Historical market data

TradingPro henter historical candles fra exchange-API'et og gemmer dem lokalt i databasen. Lokal historik gør chartet mindre afhængigt af eksterne API-kald og gør det samtidig muligt at bruge TimescaleDB til aggregering og effektiv hentning af candle-data.

Jeg valgte at gemme historical data lokalt, fordi chartet ellers skulle hente data fra exchange-API'et, hver gang brugeren skifter symbol eller bevæger sig tilbage i tiden. Det ville medføre flere eksterne requests og gøre systemet mere afhængigt af rate limits, netværksforbindelsen og midlertidige fejl hos exchange.

Når candle-data først er gemt lokalt, kan TradingPro selv styre, hvordan historikken hentes, behandles og præsenteres i chartet.

13.1 Batch-baseret synkronisering

Historical candles hentes i batches, fordi exchanges begrænser, hvor mange candles der kan returneres i et API-request. Et batch er i denne sammenhæng en mindre gruppe candles, som hentes og gemmes samlet.

Efter hvert batch beregnes det næste tidsinterval, og processen fortsætter, indtil den ønskede historik er hentet. På den måde kan TradingPro hente større mængder historical data uden at forsøge at hente hele perioden i et request.

13.2 Fortsættelse fra databasen

Ved senere synkroniseringer bruges den eksisterende historik i databasen som udgangspunkt. Systemet kan dermed fortsætte fra de allerede gemte candles i stedet for at hente den samme periode igen.

Her er OpenTime vigtig, fordi den angiver tidspunktet, hvor en candle starter. Et database-id viser kun rækkefølgen, som data er blevet gemt i, og kan derfor ikke bruges til at afgøre, hvilken candle der tidsmæssigt er den nyeste.

Dette reducerer unødvendige API-kald og gør synkroniseringen mere effektiv.

13.3 Aktivering af nye symboler

Når et nyt symbol aktiveres i TradingPro, kan det gennemgå det samme historical sync-flow som de eksisterende symboler.

Synkroniseringen er knyttet til aktive exchanges og symboler, så et nyt symbol ikke kræver ændringer i frontend for at kunne få historical data. Det gør løsningen mere dynamisk og lettere at udvide med flere markeder.

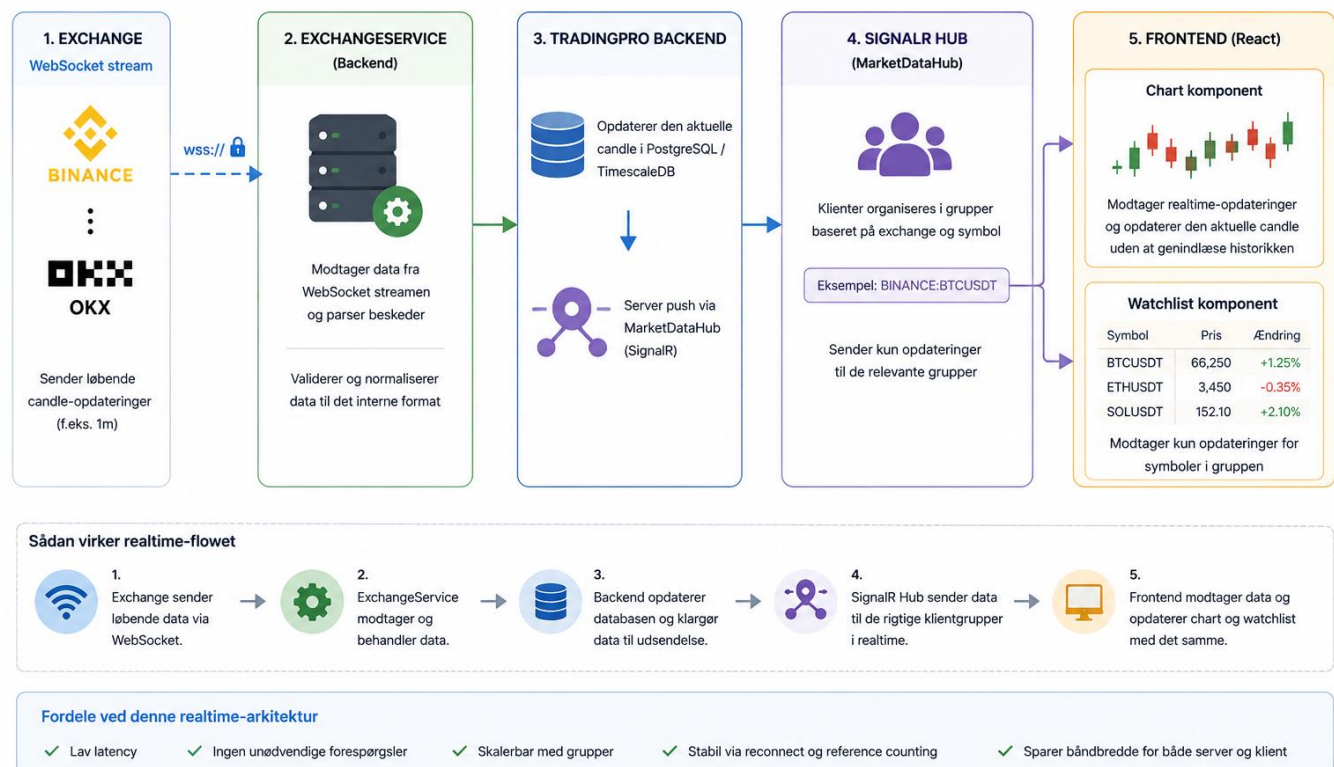
14. Realtime market data

TradingPro skal ikke kun vise historiske candles, men også kunne vise den aktuelle markedsbevægelse, mens den sker. Derfor anvender systemet realtime market data fra de exchanges, som TradingPro er integreret med.

Realtime-data modtages fra exchanges WebSocket-forbindelse. I modsætning til et almindeligt REST-kald holder WebSocket forbindelsen åben, så exchange løbende kan sende nye pris- og candle-opdateringer til backend. Backend behøver derfor ikke at spørge Binance eller OKX hvert sekund for at kontrollere, om prisen har ændret sig.

Realtime market data - flow i TradingPro

Fra exchange til brugerens chart i realtime



14.1 SignalR-grupper

Mellem TradingPros backend og frontend anvendes SignalR. MarketDataHub organiserer de tilsluttede klienter i grupper baseret på exchange og symbol, eksempelvis BINANCE:BTCUSDT.

Hvis en bruger ser BTCUSDT på Binance, tilmeldes klienten den relevante gruppe. Når backend modtager en ny opdatering for BTCUSDT, sendes den kun til klienterne i denne gruppe. En bruger, der eksempelvis følger et andet symbol, behøver derfor ikke at modtage og filtrere BTCUSDT-data.

Denne opdeling reducerer unødvendig realtime-trafik og gør det muligt for flere brugere at følge forskellige markeder samtidig.

14.2 Reconnect og reference counting

En realtime-forbindelse kan midlertidigt blive afbrudt, eksempelvis på grund af netværksproblemer eller genstart af backend. Frontendens SignalR-service holder derfor styr på de aktive abonnementer og kan tilmelde dem igen, når forbindelsen bliver genetableret.

TradingPro anvender også reference counting til abonnementerne. Det betyder i denne sammenhæng, at frontend holder styr på, hvor mange komponenter der bruger det samme realtime-abonnement.

Chartet og watchlisten kan eksempelvis begge følge BTCUSDT samtidig. Hvis chartet stopper med at bruge symbolet, skal forbindelsen ikke nødvendigvis afmeldes, fordi watchlisten stadig kan have brug for de samme realtime-data. Gruppen afmeldes derfor først, når ingen komponenter længere bruger abonnementet.

På den måde undgår systemet både unødvendige abonnementer og situationer, hvor en komponent kommer til at stoppe realtime-data for en anden.

14.3 Historical og realtime samtidig

Historical sync og realtime-data kan i nogle situationer arbejde med den samme periode samtidig. Det kan eksempelvis ske, mens systemet stadig henter ældre candles, samtidig med at WebSocket-forbindelsen modtager den nyeste candle.

Derfor er det vigtigt, at candles identificeres ud fra deres symbol, interval og OpenTime.

OpenTime beskriver, hvilken tidsperiode en candle tilhører.

Hvis realtime-flowet modtager en opdatering til en candle, som allerede eksisterer, skal den eksisterende candle opdateres i stedet for at oprette en ny logisk kopi. Det hjælper med at undgå dubletter og sikrer, at historical og realtime-data kan arbejde sammen.

Resultatet er, at TradingPro kan kombinere en stor lokal historik med løbende live-opdateringer, så brugeren både kan bevæge sig tilbage i chartet og samtidig følge den aktuelle markedsbevægelse.

15. Databaseoverblik

Databasen består både af almindelige relationelle data og store mængder market data. Jeg har valgt PostgreSQL som fælles fundament og TimescaleDB oven på PostgreSQL til candle-data. På den måde kan jeg beholde normale relationer mellem brugere, exchanges og symboler og samtidig bruge funktioner, der er lavet til tidsserier.

Område	Vigtige data	Formål
Brugere	Users og UserPreferences	Gemmer brugerens konto, rolle og personlige indstillinger.
Markeder	Exchanges og Symbols	Beskriver hvilke exchanges og handelssymboler systemet kender og kan aktivere.
Market data	Candles + TimescaleDB aggregates	Gemmer 1m-prishistorik og danner større timeframes til chartet.
Watchlists	Watchlists og WatchlistItems	Gemmer brugerens egne lister og de symboler, der er tilføjet.
Alerts	Alerts	Gemmer prisbetingelser og status for brugerens alarmer.
Chart	ChartDrawings	Gemmer brugerens tegninger, så de kan vises igen på det relevante symbol og interval.

Relationerne er vigtige, fordi data ikke står alene. Et Symbol tilhører for eksempel en Exchange, candles tilhører et Symbol, og en Watchlist tilhører en User. E/R-diagrammet i **Bilag 2** viser disse relationer grafisk, mens dette overblik forklarer, hvorfor tabellerne findes.

16. Database og Entity Framework Core

PostgreSQL bruges som den primære database. Entity Framework Core bruges til almindelige relationelle operationer, mens TimescaleDB-funktioner anvendes til market data. Denne kombination gør det muligt at have brugere, watchlists og andre relationelle data i samme databasesystem som candles.

Jeg har ikke delt almindelige applikationsdata og market data op i to forskellige databasesystemer. TimescaleDB bygger videre på PostgreSQL, så jeg kan beholde de normale relationer og samtidig få funktioner, der er lavet til tidsserier. Det gør driften enklere og giver en tydelig sammenhæng mellem Exchanges, Symbols og Candles.

16.1 E/R-diagram

E/R-diagrammet i **Bilag 2** viser blandt andet Exchanges, Symbols, Candles, Users, UserPreferences, Watchlists, WatchlistItems, Alerts og ChartDrawings. Exchanges har en 1:N-relation til Symbols. Symbols er derefter knyttet til candles og flere brugerfunktioner. Users har blandt andet relationer til watchlists og chart drawings.

16.2 Centrale tabeller

Databasen består af flere centrale tabeller, som hver gemmer forskellige typer data fra TradingPro.

Tabel	Formål
Exchanges	Registrerer understøttede exchanges og aktiv status
Symbols	Markeder/par med reference til exchange
Candles	OHLCV-data pr. symbol, interval og OpenTime
Users	Login-, rolle- og konto-oplysninger
UserPreferences	Brugerens gemte indstillinger
Watchlists / WatchlistItems	Personlige lister og deres symboler
Alerts	Prisbaserede brugeralarmer
ChartDrawings	Gemte tegninger pr. bruger, symbol og interval

17. Modeloverblik

Modellerne er de objekter, som backend bruger til at repræsentere data i programmet. De fleste modeller svarer tæt til et område i databasen, men deres vigtigste formål i koden er at give data en tydelig struktur og gøre relationerne mellem dem forståelige.

Model	Hvad den repræsenterer
User	En bruger af TradingPro med loginoplysninger, rolle og konto-relaterede data.
UserPreference	Brugerens personlige indstillinger, så oplevelsen kan bevares mellem sessioner.
Exchange	En ekstern handelsplatform som Binance eller OKX samt om den er aktiv i systemet.
Symbol	Et handelssymbol, for eksempel BTC/USDT, som er knyttet til en bestemt exchange.
Candle	OHLCV-prisdata for et symbol, interval og tidspunkt. Candle er den centrale model i chartets historiske data.
Watchlist /	En brugers egen liste og koblingen mellem listen og de symboler, der ligger i

WatchlistItem	den.
Alert	En prisregel, som brugeren opretter for et symbol, samt information om hvorvidt den er aktiv eller udløst.
ChartDrawing	En gemt tegning fra chartet sammen med symbol, interval og drawing-data.

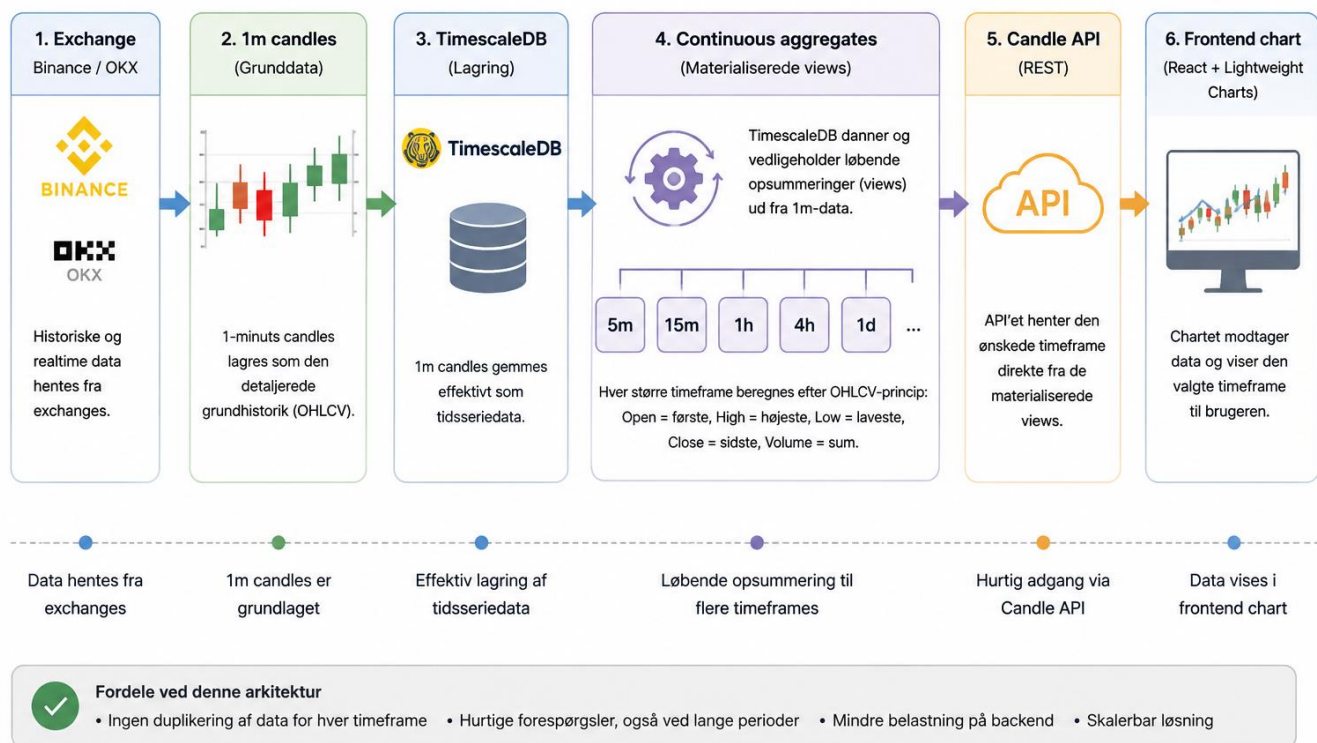
Jeg bruger desuden DTO'er, når data skal ind eller ud gennem API'et. Det er en vigtig forskel: en database-model beskriver de interne data, mens en DTO beskriver det format, som en bestemt request eller response har brug for. På den måde behøver frontend ikke at kende alle felter og relationer i de interne modeller.

18. TimescaleDB og timeframes

TradingPro gemmer store mængder candle-data, hvor tidspunktet er en central del af hver række. Hver candle indeholder blandt andet Open, High, Low, Close og Volume for et bestemt symbol og tidspunkt. Da nye candles løbende tilføjes, vokser datamængden over tid.

Derfor anvender projektet PostgreSQL sammen med TimescaleDB. TimescaleDB er velegnet til tidsseriedata og gør det muligt at arbejde effektivt med store historiske datasæt. I TradingPro bruges TimescaleDB især til lagring af candle-data og til at danne større timeframes ud fra 1-minuts historikken.

Dataflow: TimescaleDB og timeframes i TradingPro



18.1 1m som grunddata

TradingPro anvender 1-minuts candles som den detaljerede grundhistorik. En 1m candle repræsenterer prisbevægelsen inden for et minut og indeholder OHLCV-data.

Et forenklet eksempel på tre 1m candles kan være:

10:00 O=100 H=103 L=99 C=102 V=40

10:01 O=102 H=105 L=101 C=104 V=35

10:02 O=104 H=106 L=103 C=105 V=50

Når 1m-data findes i databasen, kan større timeframes dannes ud fra den samme historik. TradingPro behøver derfor ikke at gemme en separat kopi af den samme prisbevægelse for hvert interval.

1m-data fungerer dermed som projektets detaljerede grundlag, mens de større timeframes bygges oven på disse data.

18.2 Continuous aggregates

Til større timeframes anvender TradingPro TimescaleDB continuous aggregates. Et continuous aggregate fungerer som en løbende materialiseret opsummering af de underliggende 1m candles.

Hvis der eksempelvis skal dannes en 15m candle, grupperes 15 1m candles i samme tidsperiode. Den nye candle beregnes efter det normale OHLCV-princip:

- **Open** = den første Open-værdi i perioden
- **High** = den højeste High-værdi
- **Low** = den laveste Low-værdi
- **Close** = den sidste Close-værdi
- **Volume** = summen af periodens Volume

Det betyder, at 15 enkelte 1m candles kan repræsenteres som en 15m candle. Det samme princip anvendes til de øvrige timeframes i systemet.

TimescaleDB vedligeholder de materialiserede resultater, så hele datasættet ikke skal beregnes igen ved hver API-request. Candle API'et kan derfor hente den ønskede timeframe direkte fra det relevante view.

Dette er især en fordel ved lange historiske perioder. Hvis frontend eksempelvis efterspørger flere års 1d-data, behøver backend ikke først at hente et meget stort antal 1m candles og aggregere dem i C#.

Databasen kan i stedet returnere de allerede aggregerede candle-data.

18.3 Refresh-vinduer

Continuous aggregates skal opdateres, når nye eller historiske 1m candles bliver tilføjet. TradingPro anvender derfor refresh policies, som bestemmer, hvor langt tilbage TimescaleDB skal genberegne de materialiserede data.

Dette er vigtigt, fordi 1m candles godt kan eksistere i databasen, uden at de nødvendigvis allerede er blevet materialiseret i alle større timeframes.

Under udviklingen blev refresh-vinduerne derfor tilpasset, så historiske data kan dækkes op til 20 år tilbage. Det sikrer, at også ældre importerede candle-data kan indgå i de større timeframes.

På den måde fungerer TimescaleDB ikke kun som lagring af candle-data, men også som en central del af TradingPros behandling og optimering af historiske market-data.

19. Candle API og lazy loading

TradingPro kan indeholde store mængder historical candle-data, og det vil derfor være ineffektivt at hente hele historikken til browseren på en gang. Candle-API'et returnerer i stedet et begrænset antal candles ad gangen og bruger endTime til at bestemme, hvor i historikken næste data skal hentes fra.

På denne måde kan frontend hente candle-data i mindre blokke efter behov.

19.1 Første indlæsning

Når brugeren vælger et symbol og et timeframe, sender frontend et request til Candle-API'et og henter de seneste candles.

Backend finder de relevante candles i databasen og returnerer kun den ønskede mængde. Data sorteres kronologisk efter tid, så chartet modtager candles i den korrekte rækkefølge fra ældste til nyeste.

Det betyder, at chartet kan vises hurtigt uden først at hente hele den historiske dataserie.

19.2 Indlæsning mod venstre

Når brugeren bevæger chartet mod venstre for at se ældre historik, bruges tidspunktet for den ældste candle, der allerede er indlæst, som `endTime`.

Frontend sender derefter et nyt request med denne værdi, og backend returnerer den næste blok candles, som ligger før dette tidspunkt.

Flowet kan forenklet beskrives sådan:

Chart → **ældste candle** → **endTime** → **Candle API** → **database** → **ældre candles** → **chart**

På denne måde fungerer pagination ud fra candle-tid i stedet for database-id.

19.3 Hvorfor lazy loading?

Lazy loading betyder, at data først hentes, når der er behov for dem. TradingPro behøver derfor ikke sende eksempelvis flere års candle-historik til browseren, hvis brugeren kun ser de seneste candles.

Det giver hurtigere første indlæsning, lavere netværksforbrug og mindre hukommelsesforbrug i browseren. Samtidig har brugeren stadig adgang til den ældre historik ved at bevæge chartet tilbage i tiden.

20. Frontend

Frontend er den del af TradingPro, som brugeren arbejder direkte med. Den er udviklet i React og TypeScript og bygges med Vite. React gør det muligt at opdele brugergrænsefladen i mindre komponenter, mens TypeScript hjælper med at holde styr på de mange datatyper, der kommer fra REST API og SignalR. Det er især vigtigt i TradingPro, fordi chart, watchlist, alerts og andre paneler hele tiden skal reagere på nye data og på brugerens valg.

Jeg har forsøgt at holde frontend på samme måde opdelt som backend: komponenterne skal primært stå for visning og brugerinteraktion, mens API- og realtime-kommunikation ligger i services. Det betyder for eksempel, at chart-komponenten ikke selv skal kende alle detaljer om HTTPS eller SignalR-forbindelsen. Det gør det lettere at ændre kommunikationen uden samtidig at ændre hele brugergrænsefladen.

20.1 Komponentstruktur

Dashboardet består af selvstændige områder som chart, watchlist, alerts, AI-panel, søgning, brugerindstillinger og administrationsfunktioner. Jeg har opdelt dem i komponenter, så hver del primært har ansvar for sin egen visning og interaktion. Det gør koden lettere at læse og betyder, at jeg for eksempel kan ændre watchlisten uden at skulle ændre chartets interne logik. Flere komponenter kan stadig dele de samme data gennem fælles services og state, når det er nødvendigt.

20.2 Services og API-kald

HTTPS- og SignalR-kommunikation er samlet i frontend-services. Når en komponent skal hente candles, oprette en watchlist eller sende en anden request, kalder den en service i stedet for selv at bygge hele HTTPS-kaldet. Her håndteres blandt andet base URL, JWT-token, request/response og fejl. SignalR-servicen har på samme måde ansvaret for realtime-forbindelsen. Jeg valgte denne opdeling, fordi

komponenterne dermed kan fokusere på det, brugeren ser og gør, mens kommunikationen med backend ligger et samlet sted.

20.3 State og brugerens valg

Frontend holder blandt andet styr på valgt exchange, symbol, timeframe, candles, loading-status, brugerens paneler og andre valg i grænsefladen. State er vigtig, fordi en ændring et sted ofte påvirker flere dele af siden. Hvis brugeren skifter fra BTC/USDT til et andet symbol, skal chartet hente den rigtige historik, realtime-abonnementet skal flyttes til det nye symbol, og relevante paneler skal vise de nye oplysninger. Jeg har derfor været opmærksom på at rydde gamle abonnementer og data op, så events fra et tidligere symbol ikke bliver blandet ind i den aktuelle visning.

20.4 Dataflow fra backend til brugergrænsefladen

Når en side eller funktion åbnes, kommer data typisk ind i frontend på to måder. Historical data og almindelige brugerdata hentes gennem REST API, mens live market data kommer gennem SignalR. Frontend omdanner derefter response-data til de typer og strukturer, som komponenterne forventer. Chartet får for eksempel en kronologisk liste af candles, mens watchlisten kombinerer symbolinformation med de seneste realtime-priser.

Denne opdeling har været vigtig for mig, fordi historical og realtime data har forskellig levetid. Historical candles kan hentes i blokke og beholdes i chartets state, mens realtime-events skal kunne opdatere den aktuelle candle eller pris med det samme. Frontend skal derfor samle de to datakilder, så brugeren oplever dem som en sammenhængende graf.

20.5 Realtime i frontend

SignalR-forbindelsen bruges ikke kun af chartet. Watchlist og andre dele af brugergrænsefladen kan også have brug for aktuelle priser. Derfor ligger forbindelsen i en fælles service i stedet for inde i en bestemt komponent. Servicen kan tilmelde sig de relevante grupper og genoprette abonnementer efter reconnect.

Det løser også et praktisk problem: chart og watchlist kan følge det samme symbol samtidig. Frontend holder derfor styr på, om et abonnement stadig bruges af andre komponenter, før det bliver fjernet. På den måde undgår jeg, at en komponent ved lukning kommer til at stoppe realtime-data for en anden.

20.6 Loading, fejl og tomme resultater

En brugergrænseflade skal også kunne forklare, hvad der sker, når data ikke er klar. Jeg bruger derfor loading-status, tomme states og fejlbeskeder, så brugeren ikke bare møder et tomt område. Det er især relevant ved første indlæsning af candles, skift mellem symboler og API-kald til funktioner som AI-analyse eller watchlists.

Frontend skal samtidig kunne håndtere, at et request fejler uden at resten af siden går ned. Fejl fra backend behandles derfor i de relevante services og komponenter, og realtime-forbindelsen kan forsøge at forbinde igen. Det gør løsningen mere robust ved midlertidige netværks- eller API-problemer.

20.7 Design og responsivt layout

Til layout og styling bruger jeg Tailwind CSS. Det gør det muligt at bygge grænsefladen med genbrugelige utility-klasser og holde afstande, størrelser og responsive regler forholdsvis ensartede. TradingPro indeholder mange paneler og meget information på samme skærm, så det har været vigtigt at kunne justere layoutet uden at skrive store mængder separat CSS til hver komponent.

Jeg har forsøgt at holde det visuelle design mørkt og roligt, fordi chart og markedsdata skal være det primære fokus. Paneler som watchlist, alerts og AI kan åbnes ved behov, så brugeren ikke behøver at have

alle funktioner fremme samtidig. På mindre skærme kan pladsen blive mere begrænset, og derfor er komponenternes størrelse og placering lavet med responsive Tailwind-klasser, hvor det giver mening.

20.8 Sprog og genbrug af UI

Tekster i brugergrænsefladen håndteres gennem oversættelser i stedet for at blive skrevet direkte ind flere steder i komponenterne. Det gør det muligt at skifte sprog uden at kopiere selve UI-logikken. Den samme tanke bruges flere steder i frontend: fælles knapper, panelmønstre, services og datatyper genbruges, så funktioner ikke udvikler hver sin version af den samme grundlogik.

21. Candlestick-chart og analyseværktøjer

21.1 Candlesticks

Et candlestick indeholder Open, High, Low og Close for en bestemt periode. High og Low viser periodens yderpunkter. Denne form gør det muligt hurtigt at se både retning og volatilitet.

21.2 Understøttede timeframes

TradingPro understøtter 1m, 3m, 5m, 15m, 30m, 1h, 2h, 4h, 1d, 1w, 1mon og 1y. Frontend sender intervallet til API'et, og repository-laget vælger den relevante datakilde.

21.3 Chart drawings

Brugeren kan gemme egne markeringer på chartet. Tegninger forbindes med bruger, symbol og interval. Det betyder, at en tegning på BTCUSDT 15m ikke automatisk vises på en anden kombination.

22. Watchlists

Watchlists gør det muligt at samle markeder, som brugeren følger ofte. Brugeren kan oprette og slette lister og tilføje eller fjerne symboler. Watchlist-items kan vise live-pris og prisændring, så listen fungerer som et hurtigt markeds-overblik uden at alle symboler behøver åbnes i chartet.

22.1 Data og realtime

Watchlist-data gemmes relationelt i Watchlists og WatchlistItems. Realtime-priser kan genbruge SignalR-forbindelsen, hvilket undgår unødvendig polling og giver samme live-kilde som chartet.

22.2 Sletning og datakonsistens

Når et symbol fjernes fra en watchlist, slettes den konkrete WatchlistItem-relation, mens selve symbolet bevares i systemet. Hvis en hel watchlist slettes, fjernes listen og dens tilknyttede items gennem database-relationen. Der er desuden en unik kombination af WatchlistId og SymbolId, så det samme symbol ikke kan optræde flere gange i den samme liste.

23. Alerts

Alerts giver brugeren mulighed for at gemme en betingelse for et symbol, eksempelvis en target price. Datamodellen indeholder blandt andet TargetPrice, Condition, IsTriggered og IsActive. Funktionen er adskilt fra automatisk handel: en alert informerer om en betingelse, men udfører ikke en ordre.

23.1 Realtime trigger og notifikation

Aktive alerts kontrolleres mod den aktuelle pris, når backend modtager realtime market data. TradingPro understøtter betingelserne `CROSSES_UP` og `CROSSES_DOWN`. Hvis prisen opfylder betingelsen, sættes `IsTriggered` til `true` og `IsActive` til `false`, så den samme alert ikke udløses gentagne gange.

Efter en trigger sender backend eventet `ReceiveAlertTriggered` gennem `SignalR` til en bruger-specifik gruppe. Frontend kan derfor opdatere alert-listen med det samme og afspille en lokal notifikationslyd uden polling. Alert-funktionen er fortsat kun en notifikation og sender ingen handelsordre.

24. AI-analyse

AI-panelet fungerer som et supplerende analyseværktøj. Frontend sender symbol, interval og valgt sprog til backend. Backend finder de relevante data og bygger requesten til AI-tjenesten. På den måde ligger API-konfiguration og promptlogik på serversiden.

24.1 Sprog

Det valgte UI-sprog indgår i analyseflowet, så resultatet kan returneres på det sprog, brugeren arbejder med. Det reducerer risikoen for, at et engelsk interface viser en dansk eller tyrkisk analyse.

24.2 Begrænsning

AI-output skal betragtes som en forklarende vurdering baseret på de data, der gives til modellen. Funktionen kan ikke garantere fremtidige markedsbevægelser og er derfor ikke implementeret som automatisk køb/salg.

24.3 Teknisk implementering

AI-logikken er samlet i en separat `AiAnalysisService`. Servicen finder det valgte symbol og henter de seneste candles gennem repository-laget. I den aktuelle implementation hentes op til 50 candles, mens de seneste 20 bruges som OHLCV-grundlag i prompten. System- og user-prompt bygges i backend sammen med `exchange`, `symbol`, `timeframe` og valgt sprog.

Requesten sendes fra backend til Groq API med modellen `llama-3.3-70b-versatile`. API-nøglen ligger i backend-konfigurationen og eksponeres derfor ikke til browseren. Servicen håndterer også manglende symboler, utilstrækkelige candle-data, API-fejl og exceptions og returnerer en lokaliseret fejlttekst til frontend.

25. Brugerindstillinger og internationalisering

`UserPreferences` bruges til at gemme brugerrelaterede indstillinger. Frontend anvender i18n til oversættelser, så UI-tekster kan skifte sprog uden at komponenternes logik duplikeres. Indstillinger som sprog og visningsvalg kan dermed behandles som brugerpræferencer frem for hardkodede værdier.

26. Administration

Administrationsområdet giver autoriserede brugere mulighed for at vedligeholde centrale dele af systemet. Det omfatter blandt andet brugere, symboler, systemstatus og exchanges.

26.1 Exchange-administration

En exchange-række i databasen beskriver blandt andet `Code`, `Name` og `IsActive`. Det er vigtigt at skelne mellem konfiguration og integration: at oprette en vilkårlig exchange i databasen skaber ikke automatisk en API-adapter. Backend skal også have en `IExchangeService`-implementation for den pågældende kode.

Administrations-API'et er samtidig opdelt efter backendens lagdelte arkitektur: `ExchangesController` håndterer HTTP-request og response, `ExchangeManagementService` håndterer validering og

forretningsregler, og ExchangeRepository udfører forespørgsler og ændringer i databasen via AppDbContext.

26.2 Symboler

Symbols er knyttet til en exchange og indeholder blandt andet base asset, quote asset og aktiv status. Aktive symboler bruges af market-data flowet, så administration af symboler påvirker, hvilke markeder systemet skal arbejde med.

26.3 Sikker sletning og deaktivering

Administration skelner mellem fysisk sletning og deaktivering af markedsdata. En exchange kan kun slettes permanent, hvis den ikke har tilknyttede symbols. Hvis der allerede findes symbols og dermed mulig historik, returnerer API'et en konflikt i stedet for at risikere at bryde relationerne. I dette tilfælde kan exchange sættes inaktiv via IsActive.

Symbols bruger tilsvarende en aktiv status. Når status ændres, påvirker det market-data flowet, og stream-kontrollen kan genstarte realtime-streamen med den nye konfiguration. Deaktivering gør det derfor muligt at stoppe et marked uden at slette dets historiske data.

27. Authentication og sikkerhed

27.1 Login og JWT

Efter et gyldigt login udsteder backend et JWT-token. Tokenet sendes med beskyttede API-kald og kan også bruges ved SignalR-forbindelsen. Serveren kan dermed identificere brugeren uden at gemme en klassisk server-side session for hvert request.

27.2 Passwords og reset

Passwords gemmes ikke som læsbar tekst i databasen. I stedet bliver passwordet hashed med PBKDF2 og HMAC-SHA256. For hvert password genereres en tilfældig salt, og hash-funktionen køres med 100.000 iterationer. Databasen gemmer derfor salt og den beregnede hash i stedet for brugerens oprindelige password.

Når brugeren logger ind, verificeres det indtastede password ved hjælp af den gemte salt og de samme hashing-parametre. Backend kan dermed kontrollere passwordet uden at gemme det i klartekst.

Projektet indeholder også et password-reset flow med et tidsbegrænset reset-token. Der er ikke konfigureret en SMTP-mailserver i projektet, så reset-linket sendes ikke via e-mail. I den nuværende version vises linket direkte som en demonstration af reset-flowet. E-mailudsendelse kan tilføjes som en videreudvikling.

27.3 Roller

Rollebaseret adgangskontrol adskiller almindelige brugerfunktioner fra administration. Projektet arbejder med roller som User, Admin og SuperAdmin, så UI-visning og backend-autorisation kan afspejle forskellige ansvarsområder.

27.4 Soft delete og hard delete

TradingPro skelner mellem soft delete og hard delete ved sletning af brugere. Når den almindelige slettefunktion anvendes, fjernes brugeren ikke fysisk fra databasen. I stedet sættes IsDeleted til true, og tidspunktet gemmes i DeletionRequestedAt. Entity Framework Core har et globalt query filter på User, så soft-deleted brugere ikke indgår i normale forespørgsler.

Det samme princip påvirker relaterede brugerdata: query filters på blandt andet watchlists og watchlist-items skjuler data, der tilhører en soft-deleted bruger. Administrationen har desuden en separat hard-delete-funktion, som permanent fjerner brugeren fra databasen. Før brugeren fjernes, slettes den tilknyttede UserPreferences-række, hvis den findes.

Soft delete er valgt til den normale kontosletning, fordi status og sletningstidspunkt kan bevares, mens hard delete er en eksplicit administrativ handling til permanent oprydning. Denne forskel gør sletteadfærden mere kontrolleret end hvis alle DELETE-kald automatisk fjernede data fysisk.

28. Kommunikation mellem frontend og backend

TradingPro bruger primært REST API og SignalR til kommunikationen mellem React-frontend og ASP.NET Core-backend. De to teknologier har forskellige formål. REST bruges, når frontend aktivt sender en request og forventer et svar, mens SignalR bruges, når backend selv skal kunne sende realtime-opdateringer til frontend.

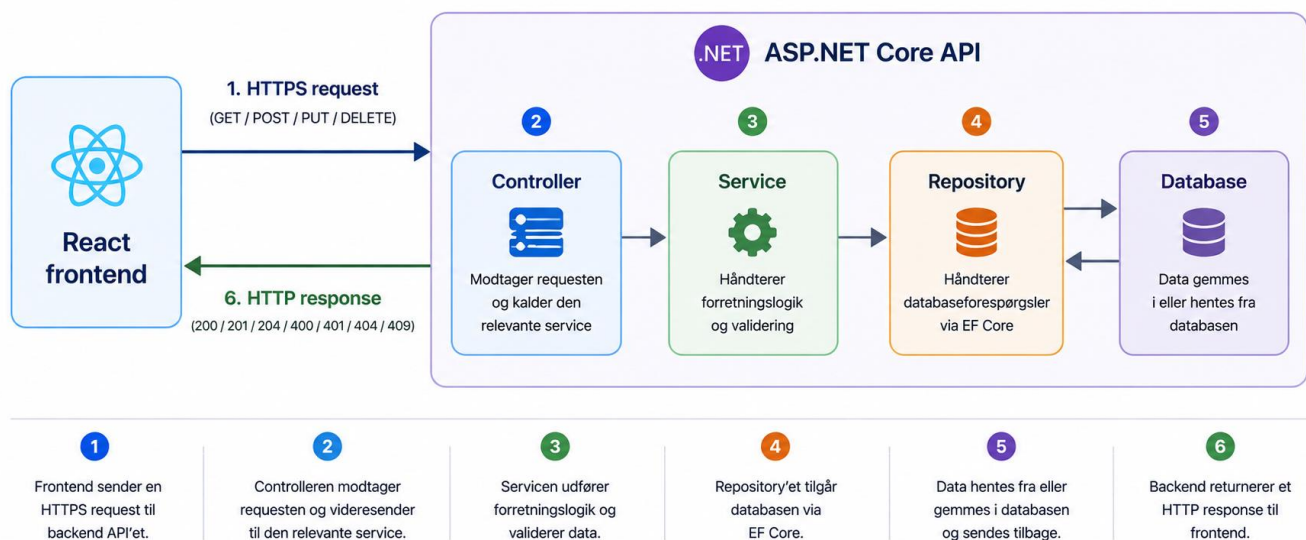
På den måde kan almindelige brugerhandlinger håndteres gennem REST, mens markedsdata kan opdateres løbende uden konstant polling.

28.1 REST API

REST API'et er den almindelige kommunikationsvej mellem React-frontend og ASP.NET Core-backend. Når brugeren eksempelvis logger ind, henter historical candles, opretter en watchlist, gemmer en alert eller ændrer en exchange i administrationen, sender frontend et HTTPS-kald til et endpoint i backend.

Frontend kommunikerer altså ikke direkte med databasen. Et request går først til backend, hvor en controller modtager requesten og sender arbejdet videre til de relevante services og repositories.

Et forenklet flow kan beskrives sådan:



Jeg bruger blandt andet GET til at hente data, POST til at oprette data eller starte handlinger, PUT til opdateringer og DELETE til sletning.

Backend returnerer samtidig HTTP-statuskoder, som fortæller frontend, hvordan requesten blev behandlet. Eksempelvis betyder 200 OK, at requesten lykkedes, 201 Created at en ressource blev oprettet og 204 No

Content, at handlingen lykkedes uden et response-body. Fejsituationer kan blandt andet returnere 400 Bad Request, 401 Unauthorized, 404 Not Found eller 409 Conflict.

Jeg har valgt at lade frontend kommunikere med TradingPros eget API i stedet for at kalde Binance og OKX direkte. På den måde ligger exchange-specifik logik, validering, authentication og databaseadgang i backend. Frontend behøver derfor kun at kende TradingPros egne endpoints og DTO'er.

28.2 SignalR

REST fungerer godt, når frontend selv starter kommunikationen, men realtime market data fungerer anderledes. Her skal backend kunne sende nye data til browseren, så snart de modtages.

TradingPro bruger derfor SignalR mellem backend og frontend. Når backend modtager en ny markedsopdatering fra en exchange, kan den sende opdateringen videre til de relevante klienter gennem MarketDataHub.

Det betyder, at frontend ikke behøver at sende et nyt REST-request hvert sekund for at spørge, om prisen har ændret sig. I stedet kan backend sende opdateringen direkte, når nye data er tilgængelige.

REST og SignalR supplerer derfor hinanden:

REST: Frontend spørger → backend svarer

SignalR: Backend modtager nye data → frontend får automatisk en opdatering

Den mere detaljerede realtime-arkitektur med WebSocket, SignalR-grupper, reconnect og reference counting er beskrevet i **afsnit 14**.

28.3 CORS og browseren

Under udvikling kan React-frontend og ASP.NET Core-backend køre på forskellige adresser eller porte. Browseren betragter dem derfor som forskellige origins.

Her anvendes CORS (Cross-Origin Resource Sharing) til at bestemme, hvilke frontend-origins der må kommunikere med backend. Backend konfigureres dermed til at acceptere requests fra den frontend, som TradingPro bruger.

Ved nogle requests sender browseren først en såkaldt preflight-request med HTTP-metoden OPTIONS. Browseren bruger denne request til at kontrollere, om den efterfølgende request er tilladt.

Et 204 No Content-svar på en OPTIONS-request kan derfor være helt normalt. Det betyder ikke nødvendigvis, at candle-requesten har returneret tomme data; det kan blot være browserens CORS-kontrol, før den egentlige request bliver sendt.

29. Fejlhåndtering og robusthed

TradingPro arbejder med flere dele, som ikke altid kan forventes at være tilgængelige hele tiden. Forbindelsen til Binance eller OKX kan eksempelvis blive afbrudt, en WebSocket-stream kan stoppe, en databaseoperation kan fejle, eller forbindelsen mellem frontend og SignalR kan midlertidigt forsvinde.

Derfor er systemet udviklet, så en midlertidig fejl så vidt muligt kan håndteres uden at stoppe hele applikationen.

Backendens background services anvender blandt andet logging og cancellation tokens. Logging gør det muligt at se, hvor og hvorfor en fejl er opstået, mens cancellation tokens bruges til kontrolleret at stoppe eller genstarte længerevarende processer.

På frontend håndteres tilsvarende situationer som loading, tomme resultater, API-fejl og tab af realtime-forbindelsen. SignalR-forbindelsen kan genetableres, og de nødvendige abonnementer kan tilmeldes igen efter reconnect.

29.1 Stream restart

Realtime market data kommer fra længerevarende WebSocket-streams til de eksterne exchanges. Disse streams skal kunne ændres, hvis TradingPros aktive symboler ændrer sig.

Hvis et nyt symbol eksempelvis aktiveres i administrationen, kan stream control stoppe den eksisterende stream kontrolleret og starte den igen med den opdaterede konfiguration. Det samme princip kan anvendes, hvis en stream skal genetableres efter en fejl.

Det betyder, at ændringer i de aktive markeder kan håndteres dynamisk uden at genstarte hele TradingPro-backend.

29.2 Datakonsistens

Historical sync og realtime-streamen kan i nogle situationer arbejde med den samme candle. Historical sync kan eksempelvis være i gang med at gemme historiske data, samtidig med at realtime-streamen modtager en opdatering for den nyeste periode.

Derfor er det vigtigt, at en candle identificeres ud fra sin logiske nøgle, blandt andet symbol, interval og OpenTime. OpenTime angiver begyndelsen på den tidsperiode, som candlen repræsenterer.

Hvis en candle for den samme periode allerede findes, skal systemet kunne opdatere den eksisterende data frem for at oprette en logisk dublet. Upsert-lignende adfærd betyder her, at data enten oprettes, hvis de ikke findes, eller opdateres, hvis de allerede eksisterer.

På den måde kan historical og realtime-data arbejde sammen, samtidig med at risikoen for dubletter og inkonsistente candle-data reduceres.

30. Teststrategi

Teststrategien i TradingPro fokuserer på de dele af backend, hvor adfærd kan kontrolleres automatisk og reproducerbart. Testprojektet er udviklet med xUnit og indeholder tests af repositories og controllers.

Jeg har især valgt at teste områder, hvor fejl kan påvirke flere funktioner i systemet. Repository-tests kontrollerer blandt andet hentning, filtrering, oprettelse, opdatering og sletning af data. Controller-tests kontrollerer, om API'et reagerer korrekt på forskellige requests og returnerer de forventede resultater og HTTP-statuskoder.

De automatiserede tests dækker ikke hele TradingPro. Funktioner som chart-interaktion, realtime-opdateringer og den samlede brugeroplevelse er derfor også afprøvet manuelt i browseren. Kombinationen af automatiserede backend-tests og manuel frontend-test giver dermed kontrol af både den interne logik og de vigtigste brugerflows.

30.1 Unit tests

En unit test kontrollerer et afgrænset stykke funktionalitet uden at starte hele applikationen.

Repository-tests anvender en isoleret databasekontekst med EF Core InMemory. Det betyder, at testene kan arbejde med midlertidige testdata uden at ændre den rigtige PostgreSQL-database.

I controller-tests kan afhængigheder erstattes med mocks, hvor det er relevant. Et mock fungerer som en kontrolleret erstatning for en rigtig dependency, så testen selv kan bestemme, hvilket resultat den afhængighed skal returnere.

Det gør testene hurtigere og gør det lettere at finde ud af, hvilken del af koden der fejler, uden at starte eksempelvis exchange-streams, SignalR eller den rigtige database.

30.2 Arrange - Act - Assert

Testene følger grundlæggende princippet Arrange – Act – Assert.

Arrange betyder, at testdata og nødvendige afhængigheder først oprettes.

Act er den handling eller metode, som testen udfører.

Assert kontrollerer til sidst, om resultatet svarer til det forventede.

Denne struktur gør testene lettere at læse, fordi det tydeligt fremgår, hvad der forberedes, hvad der testes, og hvad det forventede resultat er.

30.3 Positive og negative scenarier

Det er ikke nok kun at teste, at systemet fungerer, når input er korrekt. Derfor indeholder testene både positive og negative scenarier.

Et positivt scenarie kan eksempelvis kontrollere, at en gyldig oprettelse lykkes. Et negativt scenarie kan kontrollere ugyldigt input, manglende data, dubletter eller andre situationer, hvor API'et skal afvise en handling eller returnere en bestemt fejl.

På den måde kontrolleres ikke kun systemets normale funktionalitet, men også hvordan det reagerer, når noget ikke går som forventet.

30.4 Manuel test af frontend

Frontend er testet manuelt i browseren med fokus på de samlede brugerflows og integrationen med backend. Jeg har blandt andet afprøvet login og logout, valg af exchange og symbol, skift mellem timeframes, lazy loading af historiske candles, realtime-opdateringer, watchlists, alerts, AI-analyse, brugerindstillinger og administrative funktioner. Derudover er fejlsituationer og skift mellem realtime-abonnementer afprøvet, så gamle data ikke påvirker den aktuelle visning.

Der er ikke implementeret automatiserede frontend unit-tests i projektet. Den manuelle test er valgt, fordi en stor del af frontendens funktionalitet afhænger af samspillet mellem brugerhandlinger, REST API, SignalR og grafisk rendering. Som videreudvikling kan teststrategien suppleres med automatiserede komponent- og end-to-end-tests.

31. Testplan

Testplanen beskriver, hvilke områder der er dækket af de automatiserede tests, hvilket testmiljø der anvendes, og hvilke kriterier der bruges til at vurdere testresultatet.

31.1 Testomfang

De automatiserede tests dækker fem repositories og fire controllers.

Repository-tests fokuserer blandt andet på CRUD-operationer, filtrering, lazy-loading-relaterede forespørgsler og opdatering af data uden logiske dubletter.

Controller-tests fokuserer på API-adfærd og kontrollerer blandt andet korrekte responses ved gyldige requests samt forventet håndtering af ugyldigt input og andre fejlscenarier.

Testene er dermed målrettet centrale dele af backend, men skal ikke betragtes som en komplet test af alle komponenter og eksterne integrationer i TradingPro.

31.2 Testmiljø

Repository-tests anvender EF Core InMemory, så testdata holdes adskilt fra produktions- og udviklingsdatabasen. Hver test kan dermed arbejde med kontrollerede data uden risiko for at ændre rigtige bruger- eller market data.

Controller-tests anvender blandt andet Moq til at oprette kontrollerede mocks af services, hvor det er relevant. Det gør det muligt at teste controllerens adfærd uden at være afhængig af eksterne exchanges eller andre dele af systemet.

Testene kan derfor køres i et isoleret miljø uden at kræve en aktiv forbindelse til Binance, OKX eller den rigtige PostgreSQL-database.

31.3 Start- og afslutningskriterier

Testene kan køres, når projektet kan bygges, de nødvendige testafhængigheder er installeret, og de interfaces og komponenter, som testene afhænger af, er tilgængelige.

Et tilfredsstillende testresultat kræver, at alle planlagte automatiserede tests gennemføres uden fejl. Hvis en test fejler, skal årsagen undersøges og den relevante kode eller test rettes, før testkørslen betragtes som godkendt.

Det konkrete antal tests og resultatet af den dokumenterede testkørsel fremgår af **afsnit 32 – Testresultater**.

32. Testresultater

Bilag 3 indeholder den detaljerede testrapport med testnavne, korte forklaringer og resultat. Testene giver et sikkerhedsnet omkring de dele, der er dækket, men de erstatter ikke integrationstest af TimescaleDB, WebSocket og den samlede deployed løsning.

Testområde	Antal	Resultat
Repository tests	21	21 bestået
Controller tests	18	18 bestået
I alt	39	39 bestået - 0 fejlet

33. Performance og skalerbarhed

TradingPro arbejder både med store mængder historiske candle-data og løbende realtime-opdateringer. Performance handler derfor ikke kun om, hvor hurtigt en enkelt request bliver behandlet, men også om at undgå unødvendige databaseforespørgsler, eksterne API-kald og datatransmissioner.

Jeg har derfor forsøgt at designe dataflowet, så systemet kun henter, beregner og sender de data, der faktisk er nødvendige. De vigtigste valg er lokal historik, continuous aggregates, lazy loading og SignalR-grupper.

33.1 Lokal historik

Historical candles hentes fra exchanges og gemmes lokalt i TradingPros database. Det betyder, at frontend ikke behøver at hente den samme historik direkte fra Binance eller OKX, hver gang en bruger åbner et chart.

Når data først er synkroniseret, kan efterfølgende chart-requests i stedet hente historikken fra PostgreSQL/TimescaleDB. Det reducerer antallet af eksterne API-kald og gør systemet mindre afhængigt af exchanges rate limits, netværksforbindelse og svartider.

Lokal lagring giver samtidig TradingPro større kontrol over historikken og gør det muligt at anvende TimescaleDB, lazy loading og egne API-endpoints oven på de gemte data.

33.2 Continuous Aggregates

TradingPro gemmer 1m candles som den detaljerede grundhistorik, men brugeren kan også vælge større timeframes som 15m, 1h, 4h og 1d.

Hvis backend skulle beregne disse timeframes direkte fra alle relevante 1m candles ved hvert chart-request, ville beregningsarbejdet vokse i takt med mængden af historiske data.

Derfor anvendes TimescaleDB continuous aggregates, hvor større timeframes kan være materialiseret på forhånd. Når frontend eksempelvis efterspørger 1d-data, kan databasen hente data fra det relevante aggregate-view i stedet for at aggregere mange års 1m candles igen.

Det reducerer beregningsarbejdet ved gentagne requests og gør især lange historiske perioder mere effektive at hente.

33.3 Lazy loading

Selv om historikken ligger lokalt, vil det være ineffektivt at sende hele datasættet til browseren på en gang. Flere års 1m-data kan bestå af millioner af candles, mens brugeren typisk kun kan se en mindre del af dem på skærmen.

TradingPro anvender derfor lazy loading. Når chartet åbnes, hentes først et begrænset antal af de nyeste candles. Hvis brugeren bevæger sig længere tilbage i chartet, hentes næste blok af ældre data.

Det reducerer både mængden af data, som databasen skal returnere, netværkstrafikken mellem backend og frontend samt browserens hukommelsesforbrug. Samtidig bliver den første indlæsning hurtigere, fordi brugeren ikke behøver vente på hele historikken.

33.4 Realtime-grupper

Realtime market data kan indeholde opdateringer for mange forskellige symbols. Det vil være unødvendigt at sende alle opdateringer til alle tilsluttede brugere og derefter lade hver browser filtrere de data, den har brug for.

TradingPro anvender derfor SignalR-grupper, hvor klienterne tilmeldes grupper baseret på exchange og symbol. En bruger, der følger BINANCE:BTCUSDT, modtager dermed kun de relevante opdateringer i stedet for realtime-data for alle aktive markeder.

Det reducerer unødvendig netværkstrafik og arbejde på frontend. Samtidig gør gruppeopdelingen realtime-arkitekturen mere skalerbar, fordi backend kan målrette opdateringer til de klienter, der faktisk har brug for dem.

34. Designvalg og kodekvalitet

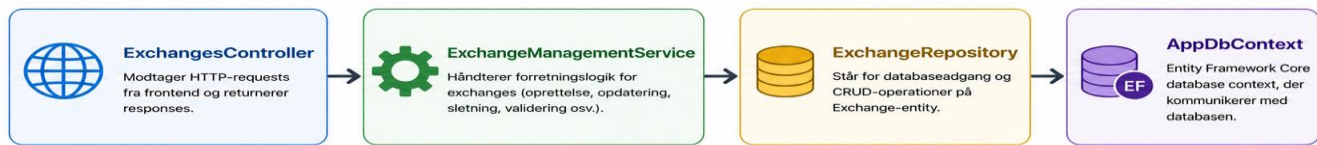
TradingPro er udviklet med fokus på, at koden skal være overskuelig og kunne ændres uden at påvirke unødvendigt mange dele af systemet. Derfor har jeg arbejdet med tydelig ansvarsfordeling, genbrugelige interfaces og DTO'er samt en struktur, hvor forskellige typer logik placeres i de lag, hvor de hører til.

34.1 Separation of concerns

Et vigtigt designvalg har været **separation of concerns**, hvor forskellige dele af systemet har forskellige ansvarsområder.

I backend modtager controllers HTTP-requests og returnerer responses, services håndterer forretningslogik, og repositories står for databaseadgang.

Et konkret eksempel er exchange-administrationen:



Det betyder, at ExchangesController ikke selv behøver at indeholde databaseforespørgsler eller regler for eksempelvis sletning af en exchange. Hvis databaseadgangen senere ændres, kan ændringen primært foretages i repository-laget uden at ændre controlleren.

Samme princip anvendes ved integrationen til eksterne exchanges. Binance- og OKX-specifik logik ligger i deres egne exchange-services, så forskelle mellem deres API'er ikke spredes ud i resten af systemet.

På frontend er kommunikationen med REST API og SignalR på samme måde samlet i services, så UI-komponenterne primært kan fokusere på visning og brugerinteraktion.

34.2 Genbrugelighed

TradingPro bruger fælles interfaces og DTO'er for at reducere afhængigheder mellem systemets dele og undgå unødvendig duplikation.

Et vigtigt eksempel er `IXchangeService`, som definerer en fælles kontrakt for exchange-integrationerne. Binance og OKX kan have forskellige endpoints, symbolformater og response-formater, men resten af TradingPro kan arbejde med dem gennem den samme grundstruktur.

Det gjorde det blandt andet lettere at tilføje OKX, fordi resten af market-data flowet ikke skulle udvikles på ny. En ny exchange kan efter samme princip tilføjes ved at implementere den fælles kontrakt og registrere den relevante service.

DTO'er bruges samtidig mellem frontend og backend, så API'et ikke behøver at eksponere database-modeller direkte. Det giver en tydeligere API-kontrakt og gør det lettere at ændre den interne implementering uden nødvendigvis at ændre frontend.

34.3 Kodekommentarer

Jeg har forsøgt at holde koden læsbar gennem tydelige klasse- og metodenavne, en ensartet projektstruktur og korte kommentarer, hvor de giver værdi.

Centrale services, repositories, controllers, background services, frontend-services og tests indeholder derfor kommentarer ved logik, som ikke er selvforklarende. Kommentarerne beskriver primært hvorfor en metode eller en bestemt del af logikken findes, frem for blot at gentage, hvad koden allerede viser.

Målet har været, at en anden udvikler skal kunne finde den relevante del af systemet og forstå dens ansvar uden først at skulle gennemgå hele projektet.

35. Brugergrænseflade og UX

TradingPro indeholder mange funktioner på samme analyse-dashboard. Brugeren skal både kunne følge et candlestick-chart, vælge exchange, symbol og timeframe samt arbejde med watchlists, alerts, AI-analyse og

andre funktioner. Derfor har det været vigtigt at holde brugergrænsefladen overskuelig, selv om systemet indeholder mange muligheder.

Jeg har valgt at gøre chartet til det centrale område, fordi analyse af prisudviklingen er TradingPro's hovedfunktion. Funktioner som watchlist og AI-analyse er placeret omkring chartet, så de kan bruges som støtte uden at fjerne fokus fra markedsdata.

Brugeren kan skifte mellem exchanges, symboler og timeframes, og brugergrænsefladen skal tydeligt vise, hvilket marked og hvilken periode der aktuelt analyseres. Målet er, at brugeren hurtigt kan navigere mellem forskellige markeder uden at miste forståelsen af de data, der vises.

35.1 Feedback og loading

TradingPro henter data fra flere forskellige kilder, og nogle handlinger kan derfor tage tid. Brugergrænsefladen skal tydeligt vise forskellen mellem loading, tomme resultater og fejl.

Når historical candles eksempelvis hentes, skal brugeren kunne se, at systemet arbejder, i stedet for blot at se et tomt chart. Hvis der ikke findes data for det valgte symbol eller timeframe, skal det kunne skelnes fra en situation, hvor et API-request er fejlet.

Det er særligt vigtigt i TradingPro, fordi et problem kan opstå flere steder i dataflowet, eksempelvis ved exchange-API'et, backend, databasen, et TimescaleDB aggregate eller kommunikationen til frontend.

Tydelig feedback gør det derfor lettere for brugeren at forstå systemets aktuelle tilstand og reducerer risikoen for, at en normal loading-situation bliver opfattet som en fejl.

35.2 Konsistens

Jeg har forsøgt at holde navigation, knapper, paneler og generelle brugerhandlinger konsistente på tværs af TradingPro. Funktioner, der udfører lignende handlinger, bør også se ud og opføre sig på en lignende måde.

Applikationen understøtter desuden flere sprog og dark theme. Sprogindstillingen gør det muligt at ændre centrale tekster i brugergrænsefladen, mens dark theme giver brugeren mulighed for at vælge den visning, der passer bedst til brugerens præference.

Fælles komponenter og central håndtering af sprog hjælper samtidig med at holde brugergrænsefladen ensartet. Det reducerer risikoen for, at forskellige dele af applikationen får forskelligt design eller viser tekster på et andet sprog end det valgte.

Samlet set har målet med UX-designet været, at TradingPro skal føles som en sammenhængende applikation, selv om funktionerne bag brugergrænsefladen arbejder med forskellige API'er, realtime-data og backend-services.

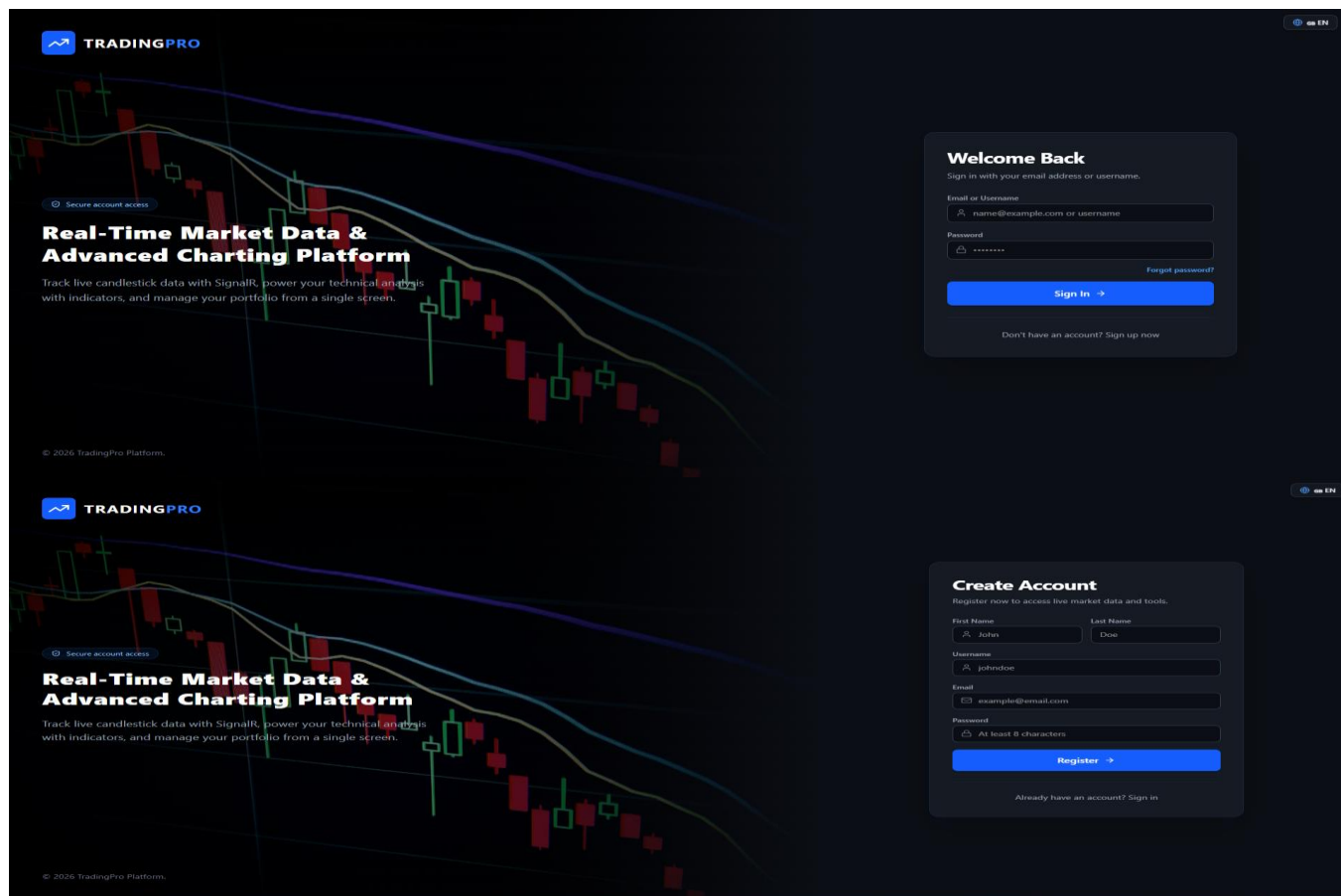
36. Brugervejledning

TradingPro er opbygget, så de mest anvendte funktioner kan nås direkte fra hovedvisningen. Brugeren vælger først exchange og symbol, hvorefter chartet viser den tilgængelige historik og fortsætter med realtime-opdateringer. De følgende afsnit beskriver de vigtigste arbejdsgange i programmet.

En mere detaljeret gennemgang af brugerfladen og de enkelte skærmbilleder findes i **Bilag 5 - Brugervejledning**.

36.1 Login og oprettelse af bruger

På login-siden indtastes brugernavn eller e-mail sammen med password. Nye brugere kan oprette en konto via registreringssiden. Efter et gyldigt login oprettes en autentificeret session, og brugeren sendes videre til hovedvisningen.



36.2 Hovedvisningen

Hovedvisningen består primært af candlestick-chartet, topmenuen og værktøjerne omkring chartet. Øverst vælges symbol, timeframe, charttype, indikatorer og layout. Sidepanelerne bruges blandt andet til watchlists, alerts og AI-analyse.



36.3 Symbol og timeframe

Symbolfeltet åbner søgningen, hvor brugeren kan finde de markeder, som er aktive på den valgte exchange. Timeframe-menuen ændrer intervallet i chartet. TradingPro understøtter både korte intervaller til detaljeret analyse og længere intervaller til overblik over større markedsbevægelser.



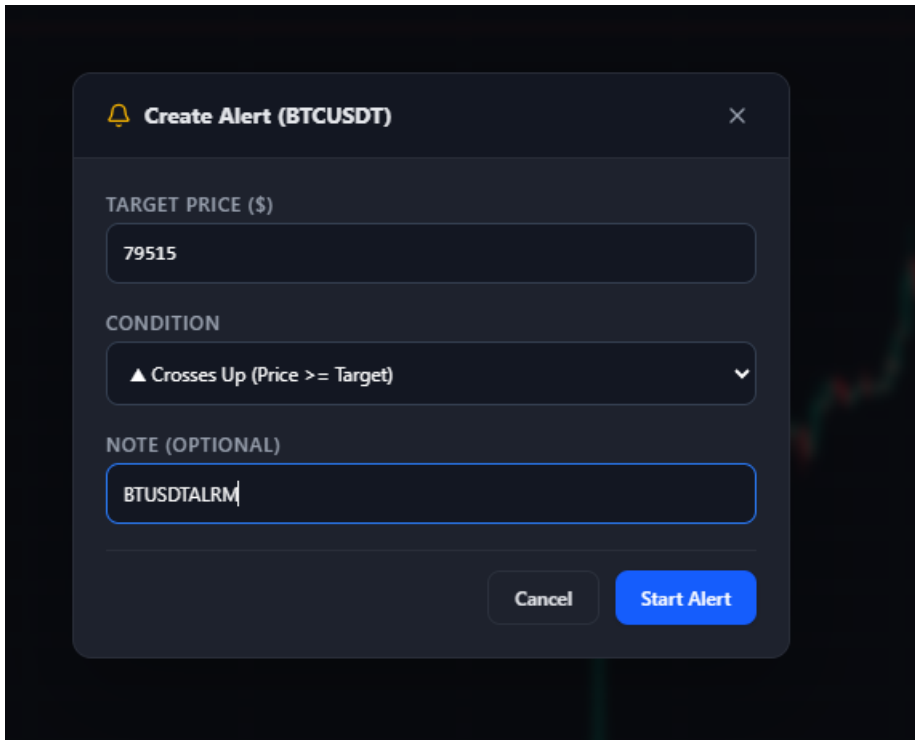
36.4 Watchlist

Watchlist-panelet bruges til at samle de symboler, som brugeren følger ofte. Der kan oprettes flere lister, og symboler kan tilføjes eller fjernes efter behov. Når et symbol modtager realtime-data, kan pris og ændring vises direkte i listen.



36.5 Alerts

Alerts bruges til at definere en pris og en betingelse for et symbol. Når markedet opfylder betingelsen, kan alerten markeres som udløst. På den måde behøver brugeren ikke konstant at overvåge chartet manuelt.



36.6 AI-analyse

AI-panelet kan bruges til at få en teknisk vurdering af det valgte symbol og timeframe. Analysen anvender den market context, som frontend sender til backend, og resultatet vises på det valgte brugergrænsefladesprog.



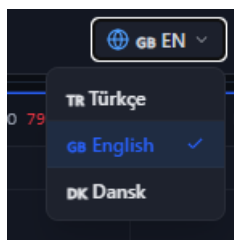
36.7 Tegneværktøjer og indikatorer

Værktøjslinjen i venstre side af chartet indeholder tegneværktøjer. Tegninger kan gemmes og knyttes til bruger, symbol og interval. Indikatormenuen bruges til at tilføje tekniske indikatorer til analysen.



36.8 Indstillinger og sprog

Brugeren kan ændre sprog og personlige indstillinger fra topmenuen og profilområdet. Sprogvalget anvendes på centrale dele af brugergrænsefladen og indgår også i AI-analyseflowet.



36.9 Administration

Brugere med administratorrolle får adgang til administrationsområdet. Her kan blandt andet users, administrators, exchanges, symbols og systemstatus administreres. Exchange-siden bruges til at styre de integrationer, som backend understøtter, eksempelvis Binance og OKX.

Management & Control Center SUPERADMIN
System users, administrator privileges, and active trading pairs

REGISTERED USERS: 1
OTHER ADMINISTRATORS: 1
ACTIVE MARKETS: 2 / 1128

System Health
Monitor the API, PostgreSQL, TimescaleDB, and market-data flow in one place.

OVERALL STATUS: Healthy
POSTGRESQL: Online
TIMESCALEDB: Online
CANDLES HYPERTABLE: Ready
ACTIVE SYMBOLS: 2 symbols

Exchange Runtime

Exchange	REALTIME	HISTORICAL	SYMBOLS	STATUS
Binance	Connected	Idle	1	Healthy
OKX	Connected	Idle	1	Healthy

LAST CANDLE
8/24/2026, 5:08:00 PM
CANDLE AGE: 79 sec
VALIDATION ERROR: 0
WARNING: 0
UPTIME: 00:11:42.1074709

37. Afprøvning og fejlsøgning

37.1 Chart viste kun få candles

En central fejl var, at nogle timeframes kun viste få candles. Fejlsøgningen viste, at et chart-problem ikke nødvendigvis er et frontend-problem. Rå 1m-data, continuous aggregate, API-response, timeframe og frontend mapping skal kontrolleres som en kæde.

37.2 Views fandtes, men data manglede

TimescaleDB jobs kunne være oprettet og stå som succesfulde, selv om et view ikke indeholdt den forventede historik. Derfor blev både policy-konfiguration og det faktiske view-indhold kontrolleret. Refresh-vinduerne blev udvidet til 20 år.

37.3 Historical og realtime rækkefølge

Når historical og realtime arbejder samtidig, kan en nyere live-candle blive indsat mellem ældre historical batches. Det demonstrerede, hvorfor OpenTime skal bruges som tidsmæssig sandhed, og hvorfor database-id ikke bør bruges som markedsrækkefølge.

37.4 Flere realtime-forbrugere

Chart og watchlist kan abonnere på samme symbol. Uden reference counting kunne en komponent afmelde SignalR-gruppen, mens den anden stadig brugte den. Abonnementsstyringen blev derfor gjort fælles.

38. Udviklingslogbog

Jeg har ført udviklingsforløbet som en uge- og dagsopdelt logbog. Planen blev justeret undervejs, når fejlsøgning eller nye tekniske behov opstod, men tabellen viser den overordnede rækkefølge i arbejdet fra uge 32 til uge 35.

Dage	Uge 32	Uge 33	Uge 34	Uge 35
Mandag	Arbejde med casebeskrivelse og projektets formål.	Opsætning af React/TypeScript frontend og grundstruktur.	Implementering af AI-analysetjeneste og analyseflow.	Afsluttende systemtest og fejlsøgning.
Tirsdag	Færdiggørelse af casebeskrivelse og afgrænsning.	Udvikling af dashboard, navigation og grundlæggende brugergrænseflade.	Integration af AI-analyse i backend og API.	Fejlrettelser, performance og optimering af produktet.
Onsdag	Aflevering af casebeskrivelse og begyndelse på kravspecifikation.	Færdiggørelse af kravspecifikation samt database- og API-plan.	Udvikling af AI-panel samt videre arbejde med watchlists og alerts.	Rapportskrivning, testdokumentation og bilag.
Torsdag	Arbejde videre med kravspecifikation og projektide.	Integration af Lightweight Charts, historical candles og chart-navigation.	Implementering af User/Admin-funktioner, exchange-administration og OKX.	Færdiggørelse af rapport, brugervejledning og gennemgang.
Fredag	Udvikling af projektstruktur og teknisk planlægning.	Implementering af timeframes, TimescaleDB og SignalR/realtime-flow.	Samlet test, fejlsøgning og rettelser i backend/frontend.	Endelig gennemgang af kode, test og samlet aflevering.

Den mere detaljerede logbog med en kort beskrivelse af hver arbejdsdag findes i **Bilag 6**. Jeg har holdt beskrivelserne korte, så logbogen viser udviklingen uden at gentage hele den tekniske rapport.

39. Projektets videreudvikling

TradingPro kan udvides på flere områder, men jeg har valgt at samle de mest relevante muligheder i fem punkter. De bygger videre på den nuværende arkitektur uden at ændre projektets grundide.

39.1 Flere exchanges

Arkitekturen med `IExchangeService` og `ExchangeServiceFactory` gør det muligt at tilføje flere exchanges efter samme princip som Binance og OKX. Det kan give brugeren adgang til flere markeder i den samme løsning.

39.2 Abonnementssystem

Et abonnementssystem kan bruges til at opdele funktioner i for eksempel en gratis og en betalt version. En premium-bruger kunne få adgang til flere alerts, flere watchlists, avancerede AI-analyser eller ekstra markedsfunktioner.

39.3 Flere analyseværktøjer

Chartet kan udvides med flere tekniske indikatorer, drawing tools og sammenligning mellem symboler. Det vil gøre TradingPro mere anvendelig til teknisk analyse.

39.4 Notifikationer

Alerts kan senere udvides med e-mail, push-notifikationer eller andre kanaler, så brugeren kan modtage besked, selv om TradingPro ikke er åben i browseren.

39.5 Deployment og skalering

En videre version kan deployes til et produktionsmiljø med central logging, overvågning og mere avanceret skalering af backend og realtime-forbindelser. Det bliver især relevant, hvis mange brugere følger market data samtidig.

40. Konklusion

I dette projekt har jeg udviklet TradingPro som en samlet webapplikation til visning og analyse af markedsdato. Målet var ikke kun at få et candlestick-chart til at fungere, men at udvikle en løsning, hvor market data kan hentes fra flere exchanges, gemmes som historik, vises i forskellige timeframes og samtidig opdateres realtime i brugergrænsefladen. Projektet har derfor kombineret backend, database, API-integration, realtime-kommunikation, frontend, sikkerhed og test.

I forhold til problemformuleringen vurderer jeg, at de centrale mål er blevet opfyldt. TradingPro kan hente market data fra Binance og OKX, gemme historical candles lokalt og vise dem i et interaktivt chart. Systemet understøtter forskellige timeframes, lazy loading af ældre historik og realtime-opdateringer. Derudover indeholder løsningen blandt andet brugerhåndtering, watchlists, alerts, chart drawings, AI-analyse og administrative funktioner.

En af de største udfordringer var at kombinere historical og realtime-data i det samme chart. Historical candles kommer fra databasen, mens realtime-data kommer løbende fra exchange. Hvis de to dataflows ikke håndteres korrekt, kan der opstå dubletter eller forskellige versioner af den samme candle. Løsningen blev at bruge WebSocket mellem exchange og backend og SignalR mellem backend og frontend. Candles identificeres samtidig ud fra blandt andet symbol, interval og OpenTime, så en realtime-opdatering kan opdatere en eksisterende candle i stedet for at oprette en dublet. Resultatet er, at brugeren oplever historical og realtime-data som en sammenhængende dataserie i chartet.

En anden udfordring var håndteringen af store mængder historical candle-data. Flere års 1m-data kan bestå af meget store datasæt, og det ville være ineffektivt at sende hele historikken til browseren eller beregne større timeframes igen ved hvert request. Løsningen blev at gemme candle-data lokalt i PostgreSQL og anvende TimescaleDB til tidsseriedata og continuous aggregates. Større timeframes kan dermed bygges oven på de detaljerede 1m-data. Samtidig anvender frontend lazy loading, så chartet først henter de seneste candles og derefter ældre data i mindre blokke, når brugeren bevæger sig tilbage i tiden. Det reducerer mængden af data, som databasen, netværket og browseren skal håndtere på en gang.

En tredje udfordring var forskellene mellem Binance og OKX. De to exchanges bruger forskellige API-endpoints, symbolformater og response-formater. Hvis disse forskelle blev håndteret direkte i controllers, repositories eller frontend, ville resten af systemet blive tæt afhængigt af den enkelte exchange. Løsningen blev at placere exchange-specifik logik bag en fælles IExchangeService-kontrakt. ExchangeServiceFactory vælger den korrekte implementation ud fra den valgte exchange, mens Binance- og OKX-servicen håndterer deres egne forskelle. Da OKX blev tilføjet, kunne jeg derfor genbruge den eksisterende struktur uden at omskrive hele market-data flowet.

Et andet vigtigt område var backend-arkitekturen. Efterhånden som projektet fik flere funktioner, blev det vigtigt at undgå, at controllers både håndterede requests, forretningslogik og databaseadgang. Løsningen har været at opdele backend i controllers, services og repositories med tydelige ansvarsområder. Et konkret eksempel er exchange-administrationen, hvor flowet går fra ExchangesController gennem ExchangeManagementService og ExchangeRepository til AppDbContext. Det gør koden mere overskuelig og gør de enkelte dele lettere at vedligeholde, teste og videreudvikle.

På frontend-siden var en udfordring at samle mange funktioner uden at gøre brugergrænsefladen uoverskuelig. Brugeren skal både kunne arbejde med chart, exchange, symbol, timeframe, watchlists, alerts, drawings og AI-analyse. Løsningen blev at lade chartet være det centrale område og placere de øvrige funktioner omkring det. React, TypeScript, Tailwind CSS og Lightweight Charts er blevet brugt til at opbygge

brugergrænsefladen. Historical data hentes gennem REST API'et, mens realtime-opdateringer modtages gennem SignalR. Målet har været, at brugeren oplever de forskellige teknologier som en sammenhængende applikation.

En del af problemformuleringen handlede også om integration af eksterne AI-tjenester uden at gøre resten af systemet afhængigt af en bestemt leverandør. AI-funktionen er derfor holdt adskilt fra exchange-integrationen og det centrale market-data flow. AI bruges som et støtteværktøj til teknisk analyse af udvalgte markedsdata, men TradingPro er ikke afhængigt af AI-tjenesten for at kunne hente, gemme eller vise market data. Denne opdeling gør det lettere senere at ændre eller udskifte den eksterne AI-tjeneste.

Authentication og rollebaseret adgangskontrol var ligeledes en del af problemformuleringen. Her var målet at sikre, at ikke alle brugere har adgang til de samme funktioner. Løsningen blev JWT-baseret authentication kombineret med rollerne User, Admin og SuperAdmin. Backend kan dermed kontrollere både, om en bruger er logget ind, og om brugeren har den nødvendige rolle til en bestemt funktion. Almindelige brugerfunktioner og administrative funktioner kan derfor holdes adskilt, og sikkerheden afhænger ikke kun af, hvilke knapper eller sider der vises i frontend.

Test har været med til at kontrollere de vigtigste dele af backend. Repository- og controller-tests kontrollerer blandt andet dataadgang, filtrering, CRUD-operationer, validering og API-adfærd. Frontend og realtime-funktionalitet er desuden blevet afprøvet manuelt gennem de vigtigste brugerflows. Arbejdet med test har også vist mig, at testbarhed hænger tæt sammen med arkitektur, fordi klare interfaces og adskilte ansvarsområder gør de enkelte dele lettere at teste.

Projektet har samtidig vist nogle områder, som kan videreudvikles. Frontend kan eksempelvis få automatiserede komponent- og end-to-end tests. Password-reset flowet kan udvides med en SMTP-mailservice, så reset-linket kan sendes direkte til brugeren. Derudover kan flere exchanges og eksterne tjenester senere tilføjes gennem den eksisterende arkitektur.

Samlet set vurderer jeg, at TradingPro besvarer de spørgsmål, der blev stillet i problemformuleringen. Projektet viser, hvordan historical og realtime market data kan håndteres gennem en struktureret backend- og databasearkitektur, hvordan markedsdata kan præsenteres i en React-baseret brugergrænseflade med interaktive charts og forskellige timesteps, hvordan exchanges og AI-tjenester kan integreres med begrænset afhængighed af den enkelte leverandør, og hvordan authentication og rollebaseret adgangskontrol kan beskytte bruger- og administrationsfunktionerne.

Den største læring for mig har ikke kun været at få de enkelte teknologier til at fungere, men at få dem til at arbejde sammen som et samlet system. Jeg har især fået mere erfaring med backend-arkitektur, API-integration, tidsseriedata, realtime-kommunikation, databasearbejde, frontend, sikkerhed og test. Samtidig har projektet givet mig en bedre forståelse for, hvor vigtigt det er at strukturere en større løsning med tydelige ansvarsområder og et gennemtænkt dataflow, så systemet både kan vedligeholdes og videreudvikles.

41. Bilagsoversigt

Bilag	Indhold
Bilag 1 - Kravspecifikation	Oprindelige krav, use cases og afgrænsning
Bilag 2 - Diagrammer	Systemarkitektur, market-data flow, login-flow, datamodel og E/R-diagram
Bilag 3 - Test Rapport	Teststrategi, 39 tests, forklaringer og resultater
Bilag 4 - Kodeindeksering	Kodeindeks med filforklaringer og kildekode fra backend, frontend og testprojekt
Bilag 5 - Brugervejledning	Brugervejledning
Bilag 6 - Udviklingslogbog	Udviklingslogbog